



操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



第十五讲 — 典型文件操作及系统





中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



典型文件系统操作



中山大學

SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



文件系统

User Space

Processes

Operating System
Kernel

File system
Implementation

FAT32, EXT2/3
KV, Distributed FS,
Graph System...

File System Operations

Devices





中山大學

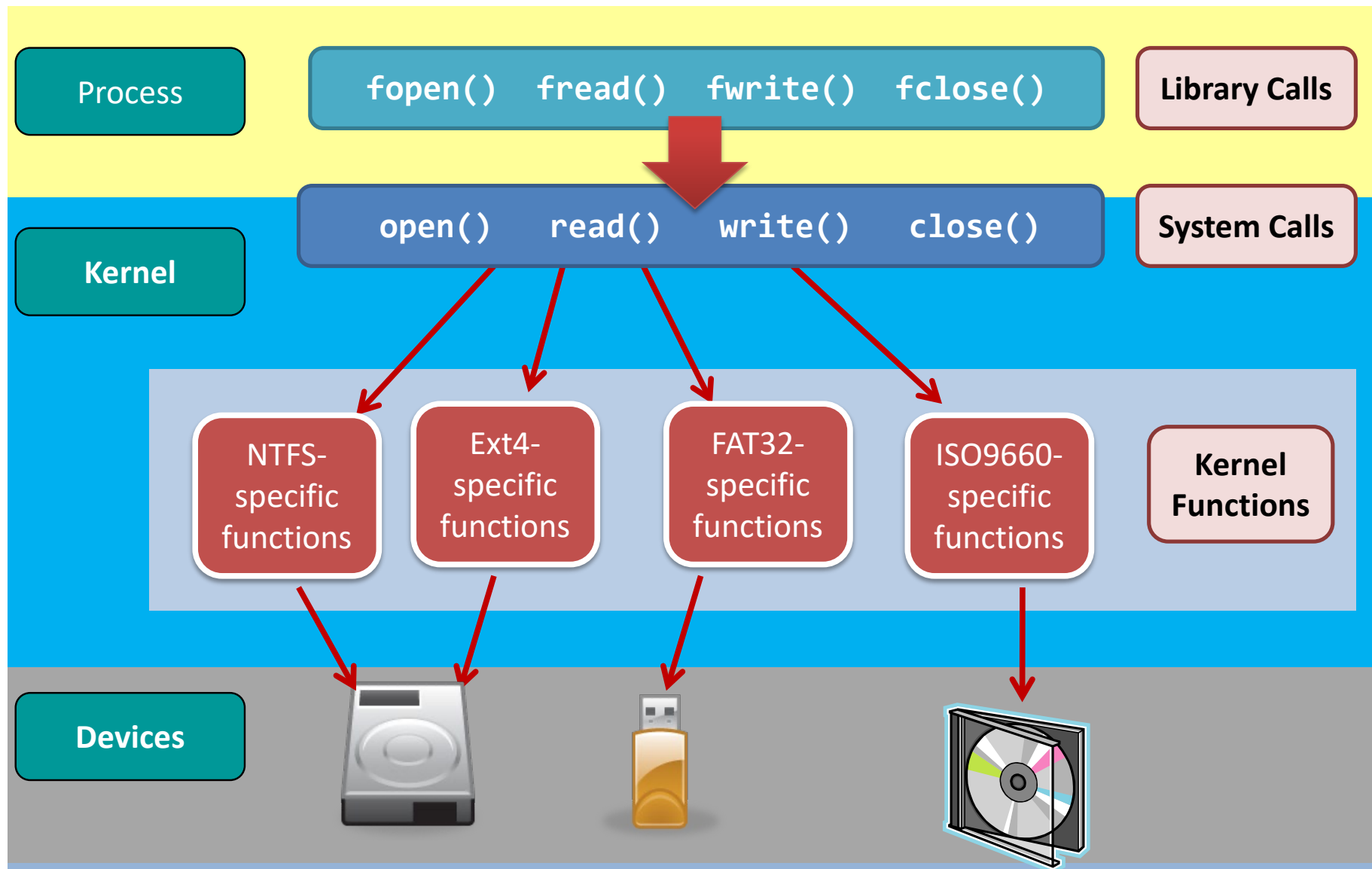
计算机学院 (软件学院)

SUN YAT-SEN UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



文件系统



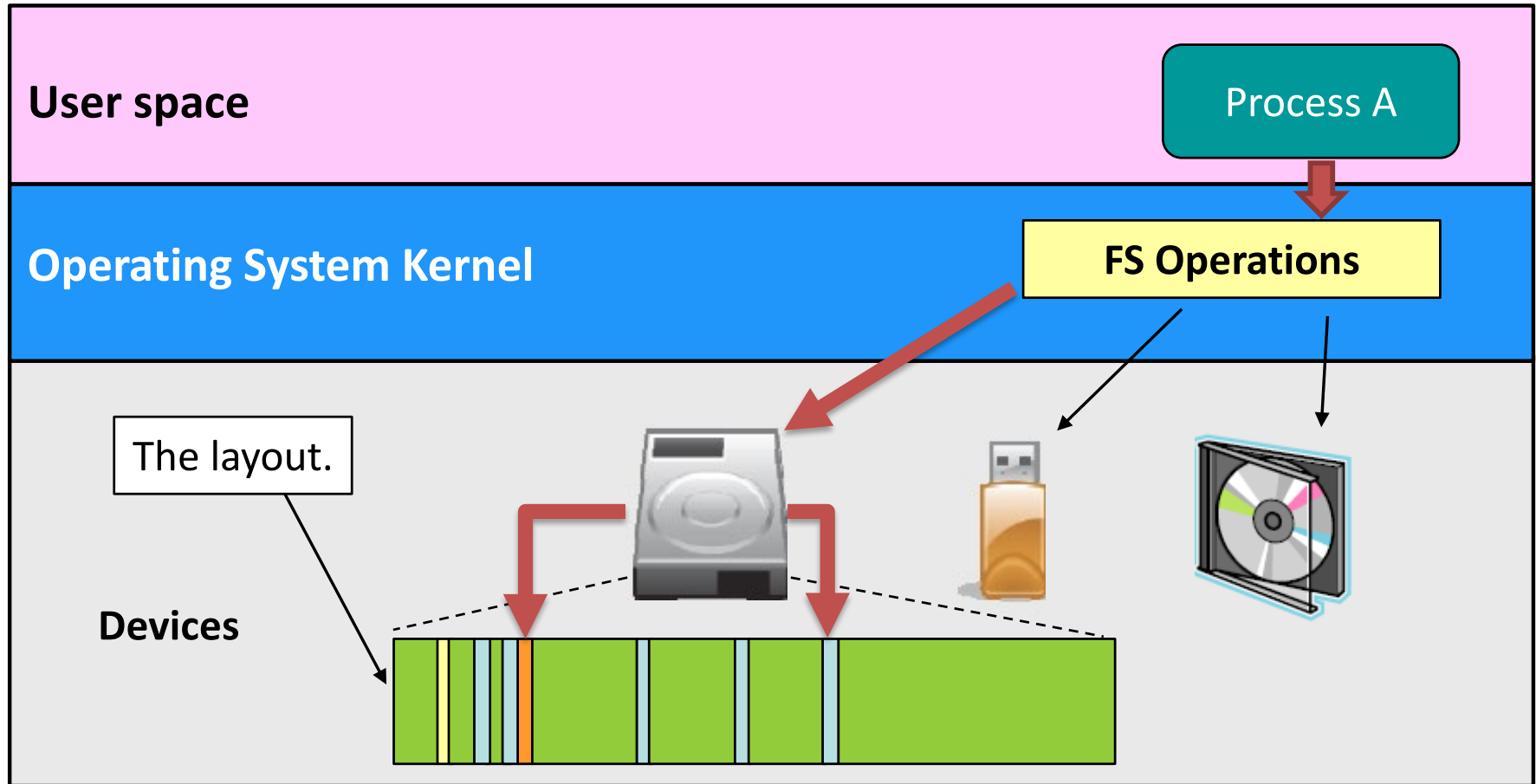


中山大學

SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



The process uses **system calls**, which then invoke the FS operations, to access the storage device.



File – what are going to be stored?

- E.g., a text file.

```
h e l l o _ w o r l d '\n'
```

test.txt

What can we find out in this example?

Content?	Content of the file
Filename?	Content of its parent directory
File size?	Attribute of the file

When a file is named, it becomes independent of the process, the user, and even the system



File Attributes

- Typical file attributes

Name	Human-readable form
Identifier	Unique tag (a number which identifies the file within the FS)
Type	Text file, source file, executable file...
Location	Pointer to a device and to the location of the file on the device
Size	Number of bytes, words, or blocks
Time, date	Creation, last modification, last use...
Protection	Access control information (read/write/execute)

You can try the command “ls -l”



File Attributes

- File attributes are FS dependent.
 - Not OS dependent.

The design of FAT32 does not include any security ingredients.

Common Attributes	FAT32	NTFS	Ext2/3/4
Name	✓	✓	✓
Size	✓	✓	✓
Permission		✓	✓
Owner		✓	✓
Access, creation, modification time	✓	✓	✓



File Permissions

- E.g., in Unix system

-rw-rw-r--	1	pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5	pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2	pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2	jwg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1	pbg	staff	9423	Feb 24 2012	program.c
-rwxr-xr-x	1	pbg	staff	20471	Feb 24 2012	program
drwx--x--x	4	tag	faculty	512	Jul 31 10:31	lib/
drwx-----	3	pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3	pbg	staff	512	Jul 8 09:35	test/

First field: File/director

2nd /3rd /4th fields (3 bits each): controls read/write/execute for the file owner/file's group/others (e.g., 111:7,110:6)

What is the meaning of the permission 775/664?



Writing attributes?

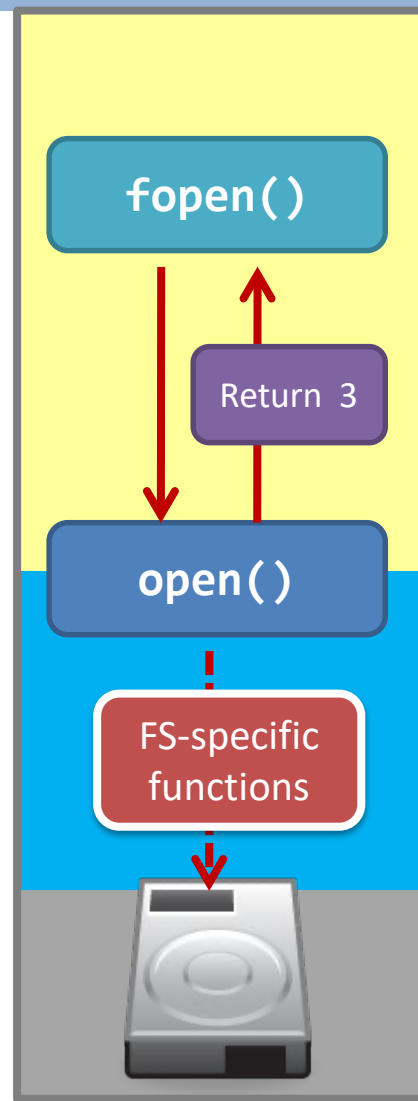
- Can you change those attributes directly?

Common Attributes	Way to change them?	
	Command?	Syscall?
Name	mv	rename()
Size	Too many tools to update files' contents	write(), truncate(), etc.
Permission	chmod	chmod()
Owner	chown	chown()
Access, creation, modification time	touch	utime()



File Open – Example

- What is **fopen()**?
 - First thing first, **fopen()** calls **open()**.
 - `FILE *fopen(const char *filename, const char *mode)`
- What is the type “**FILE**”?
 - “**FILE**”: a structure defined in “**stdio.h**”.
 - **fopen()** creates memory for the “**FILE**” structure.
 - Fact: occupying space in the area of dynamically allocated memory, i.e., `malloc()`





What is inside the “FILE” structure?

- There is a lot of helpful data in **FILE**:
 - Two important things: the **file descriptor** and **a buffer**!

```
int main(void) {  
    printf("fd of stdin  = %d\n", fileno(stdin) );  
    printf("fd of stdout = %d\n", fileno(stdout) );  
    printf("fd of stderr = %d\n", fileno(stderr) );  
}
```

fileno() returns the file descriptor of the FILE structure.

The type of **stdin**, **stdout**, and **stderr** is “FILE *”

```
$ ./fileno  
fd of stdin  = 0  
fd of stdout = 1  
fd of stderr = 2  
$_
```



File operations

- The operating system should provide...

Create

Allocate space, add an entry in the directory

Write

Filename, file content (write pointer)

Read

Filename, mem location (read pointer)

Reposition

File seek (not involve actual I/O), **required for random accesses**

Delete

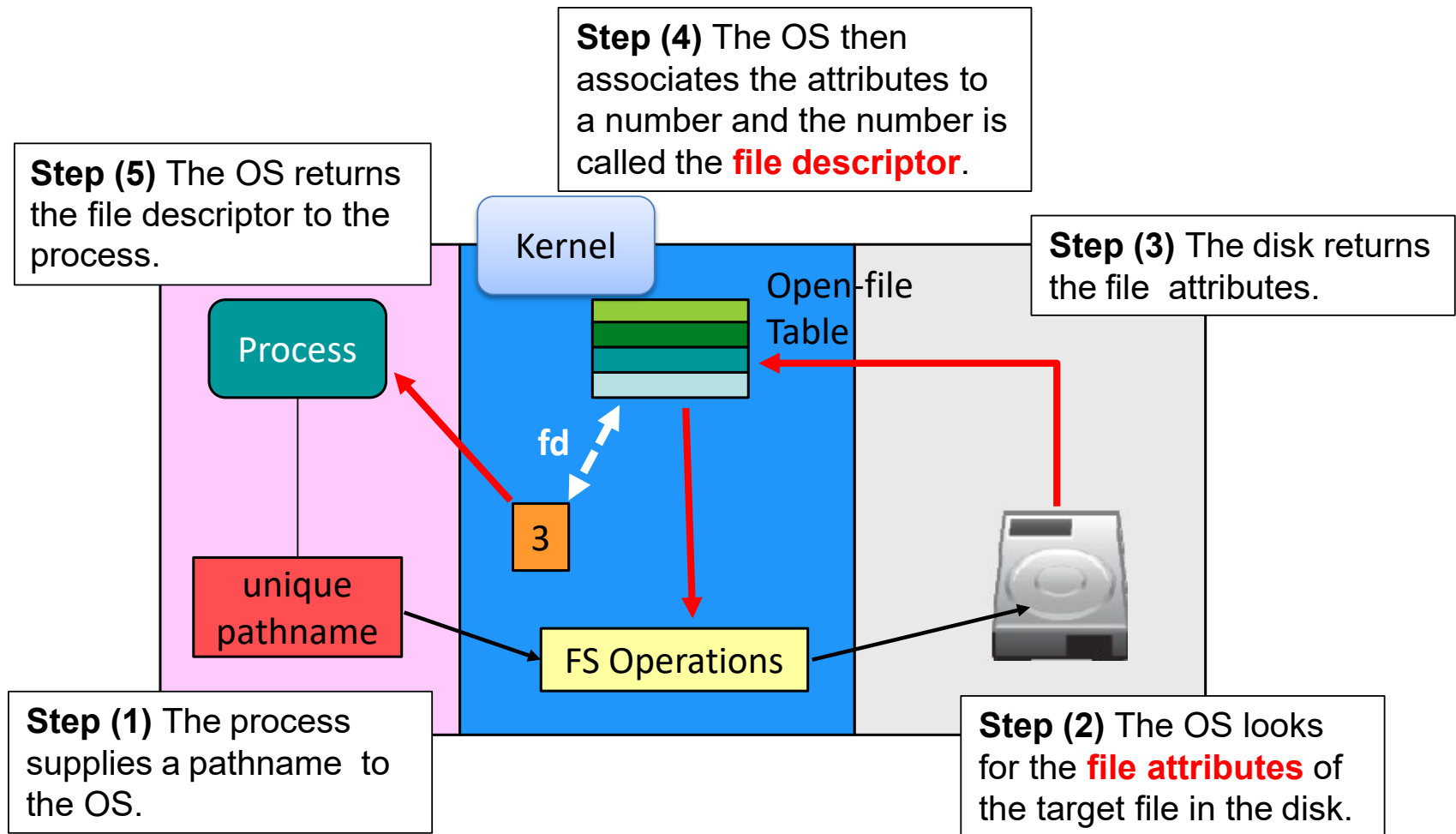
Release space, and erase directory entry

Truncate

Keeps attributes only



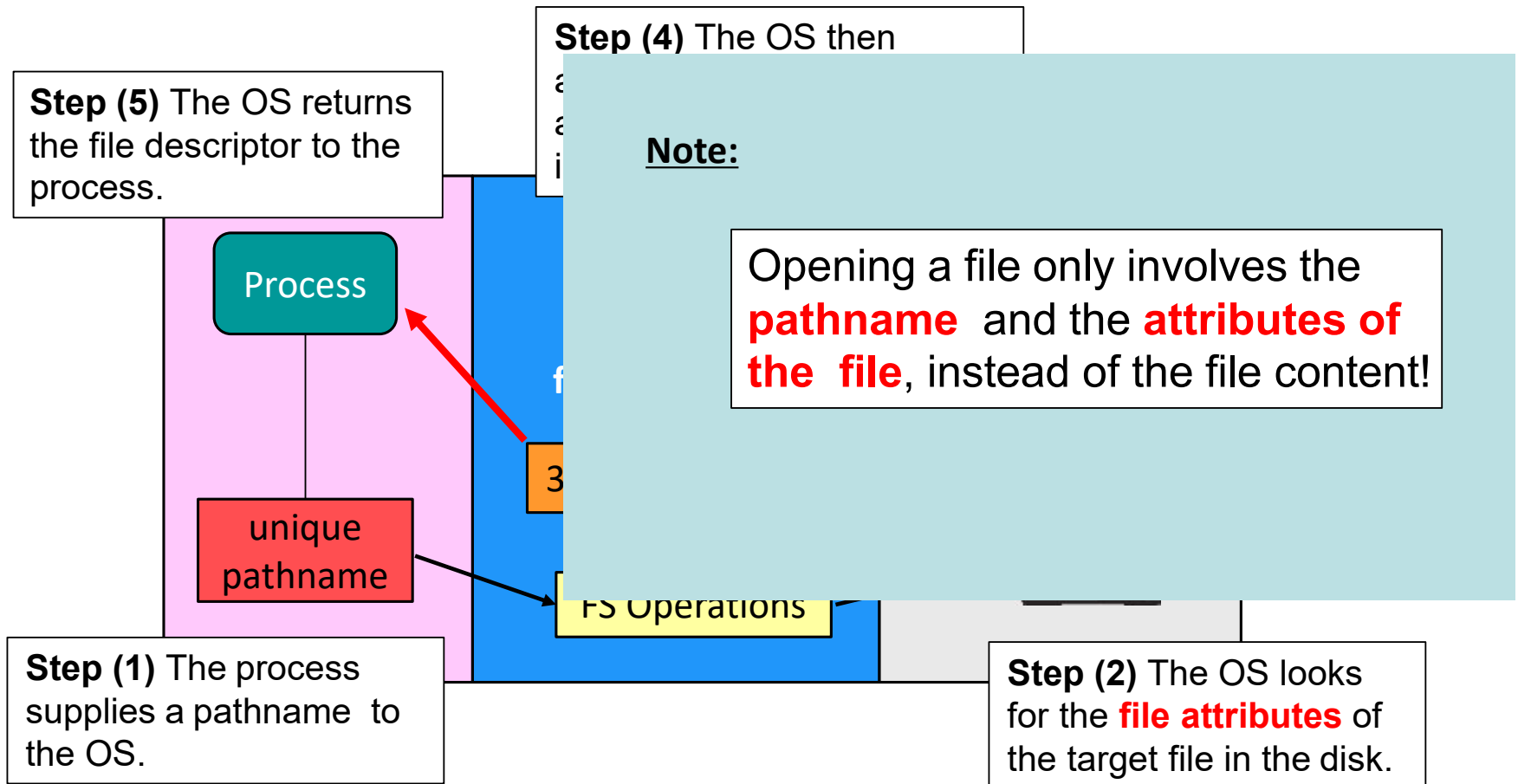
The Truth of Opening a File



Note: these steps are OS-independent as well as FS-independent.



The Truth of Opening a File



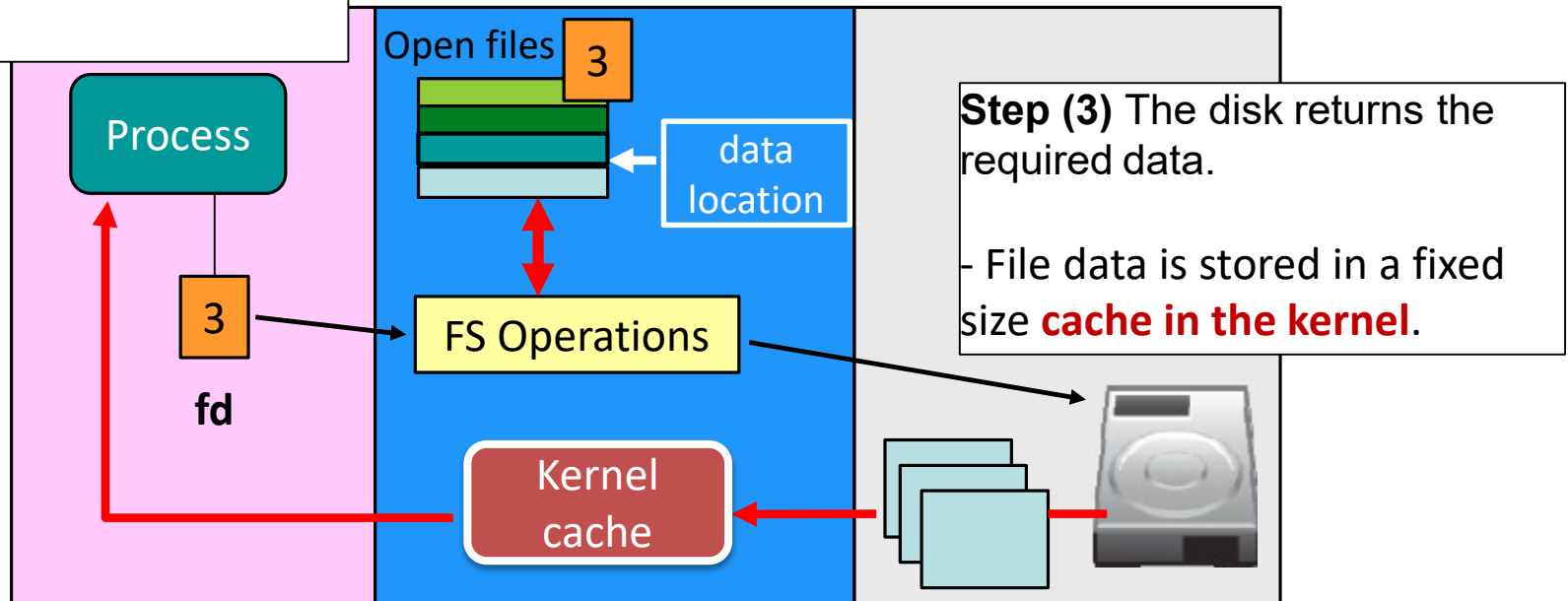
Note: these steps are OS-independent as well as FS-independent.



How to read from open files

Step (1) The process supplies a file descriptor to the OS.

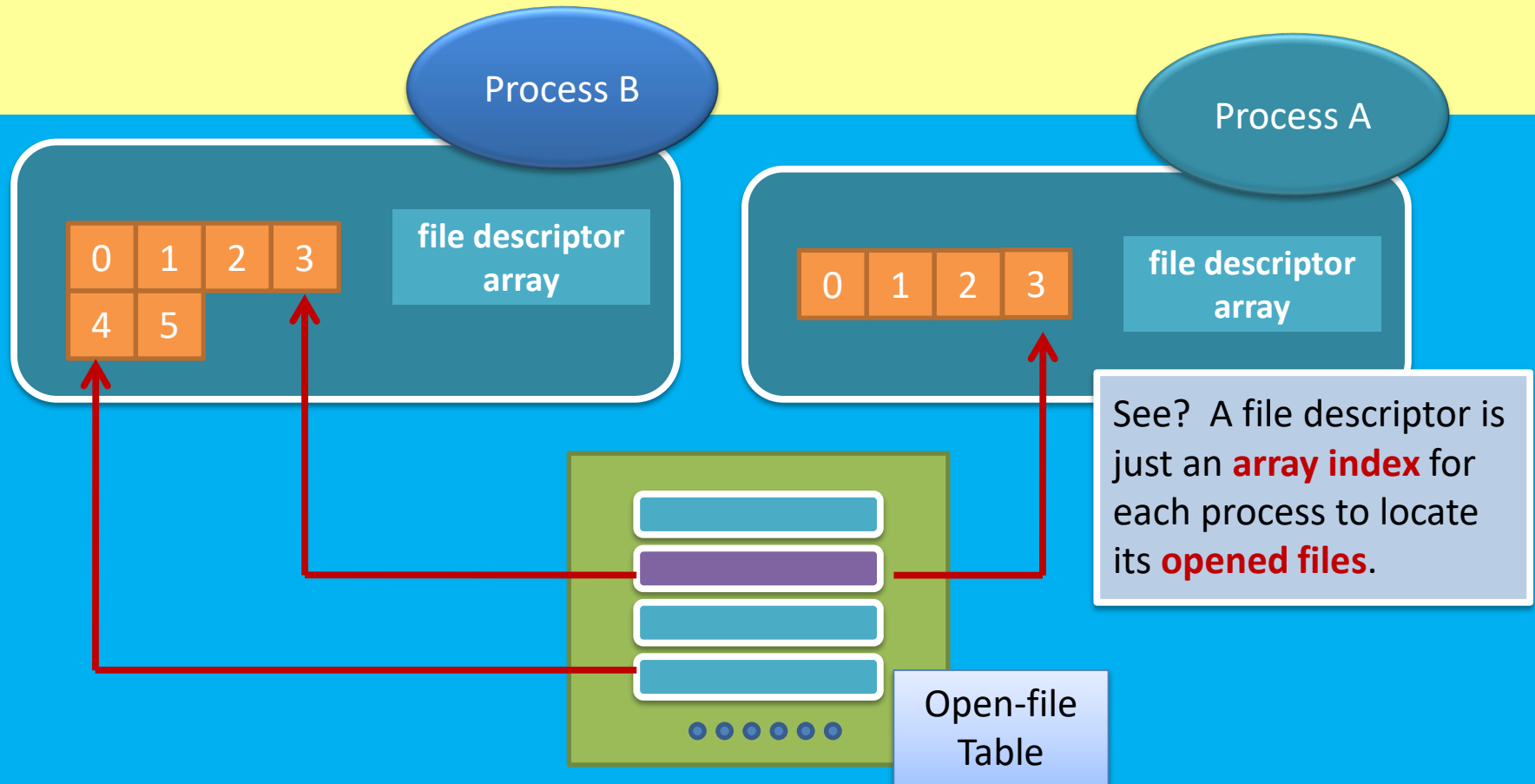
Step (2) The OS reads the file attributes and uses the stored attributes to **locate the required data**.



Step (4) The OS fills the buffer provided by the process with the data. **Write data to the userspace buffer.**



What is a file descriptor?



Although a file is opened by two different processes, the kernel uses **one structure to maintain it!**





read() & write()

Library calls that eventually invoke the read() system call	Library calls that eventually invoke the write() system call
scanf(), fscanf()	printf(), fprintf()
getchar(), fgetc()	putchar(), fputc()
gets(), fgets()	puts(), fputs()
fread()	fwrite()

- You know, I/O-related calls will invoke system calls.

```
int read ( int fd, void *buffer, int bytes_to_read )
```

From file to buffer.

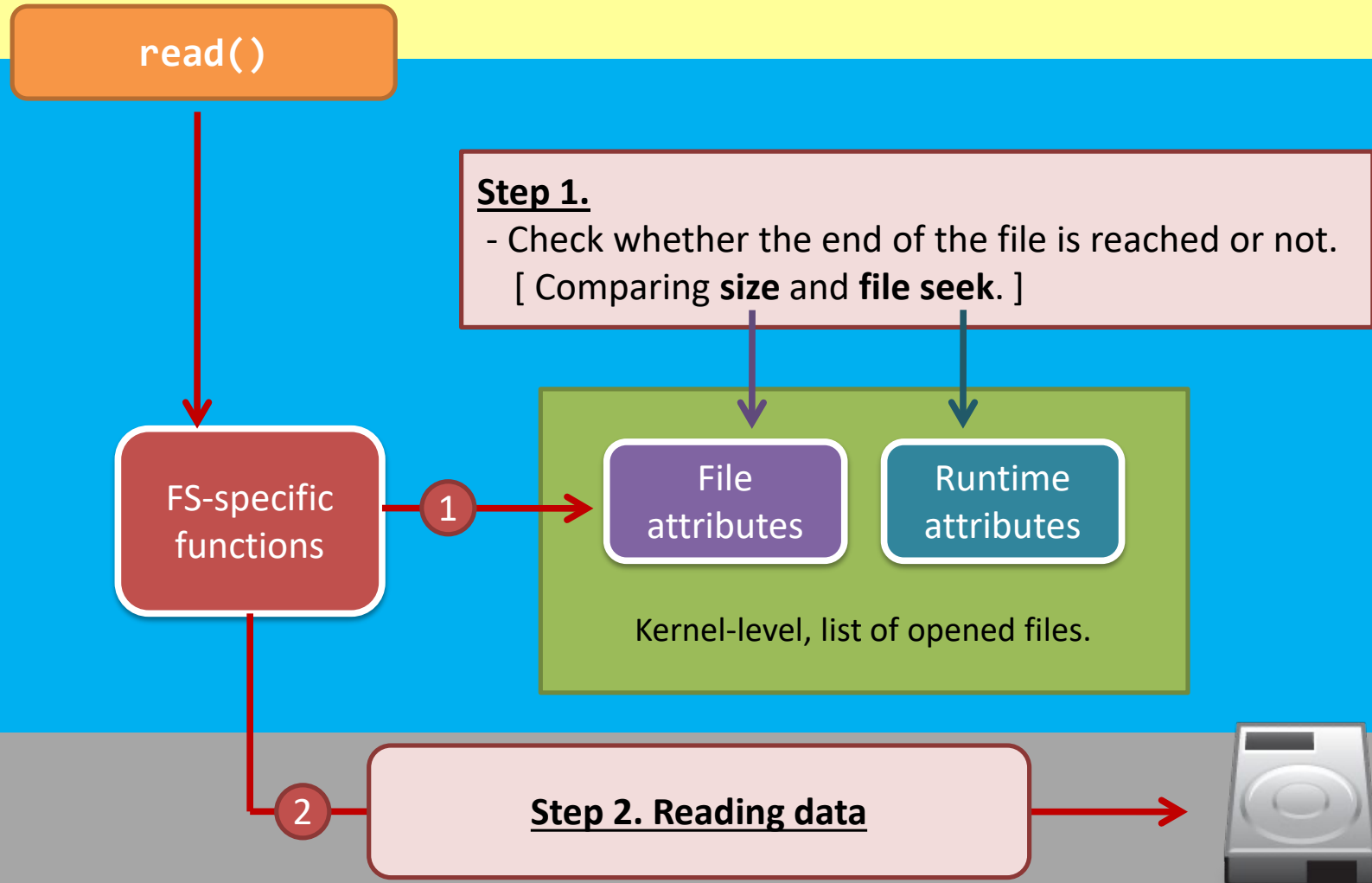
```
int write ( int fd, void *buffer, int bytes_to_write )
```

From buffer to file.

Note: I modified the function prototypes.

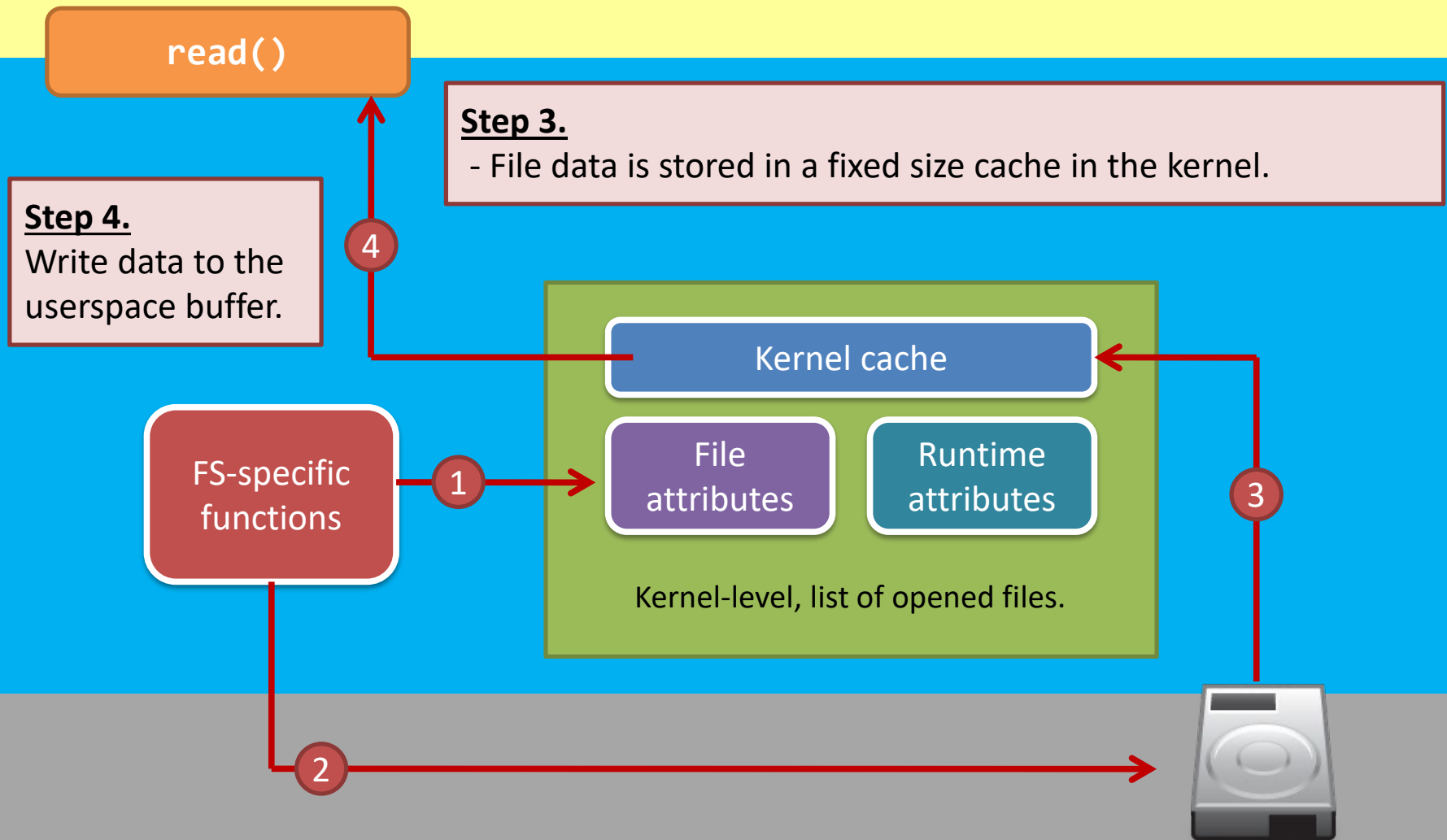


read() system call



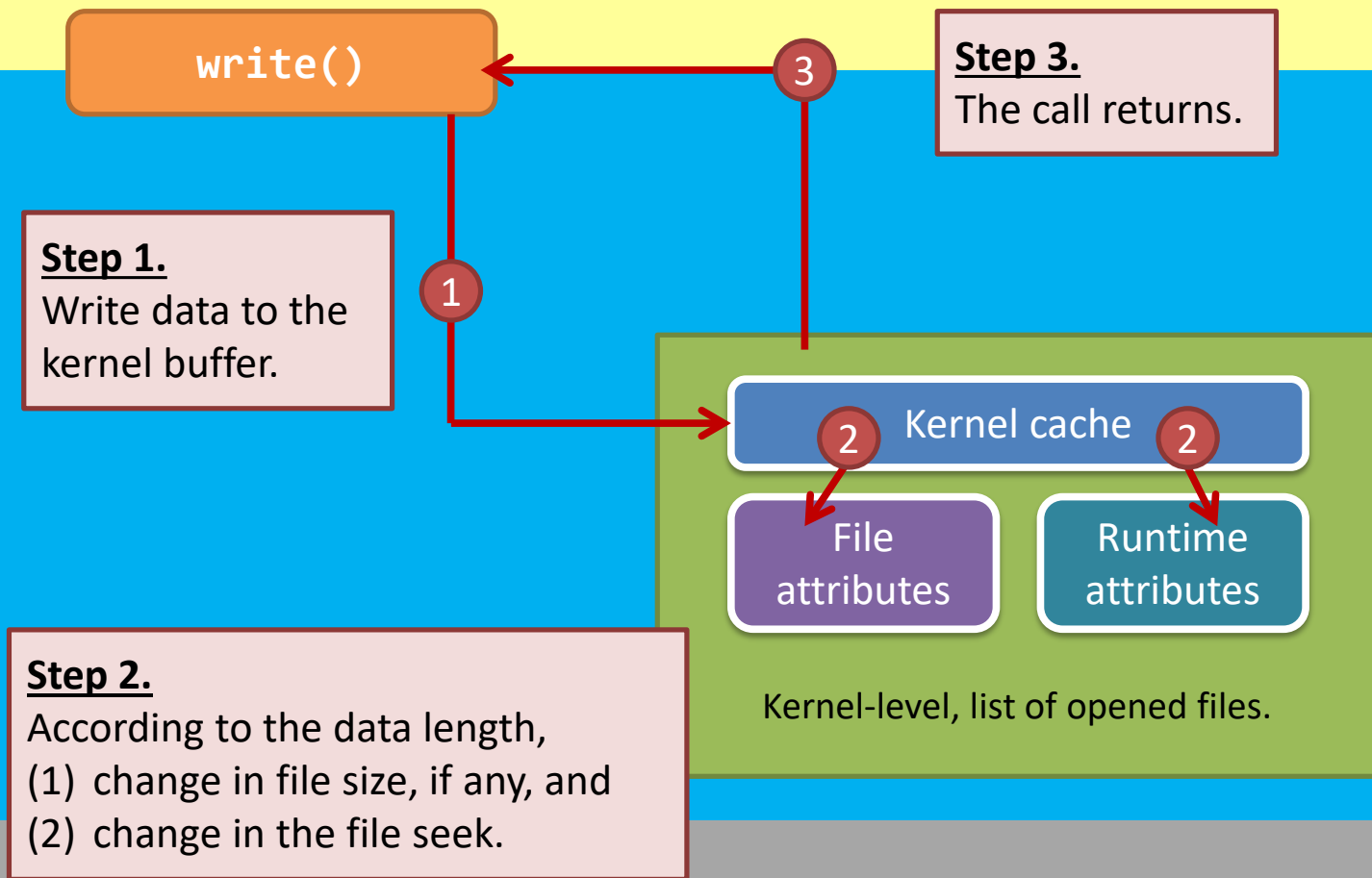


read() system call



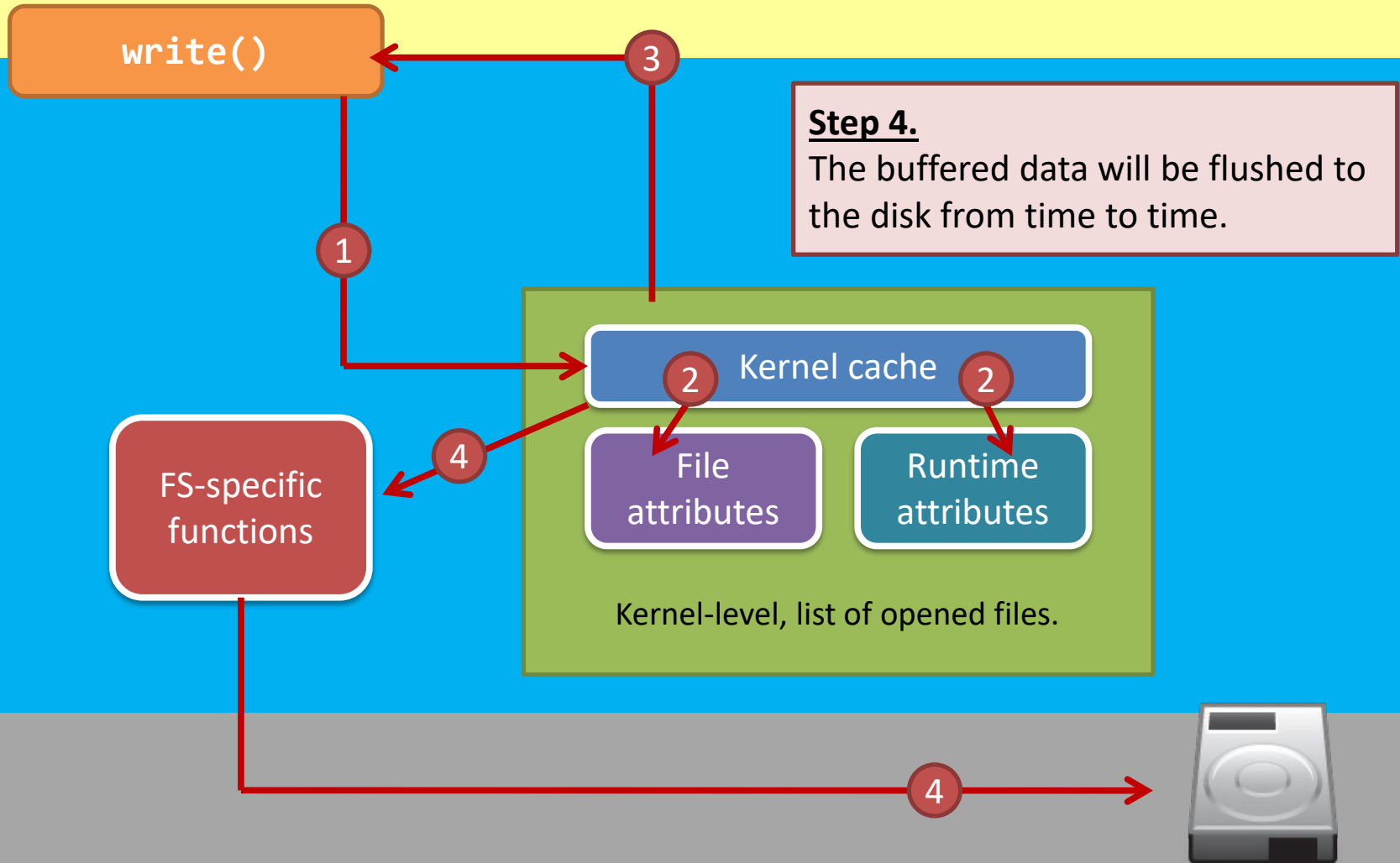


write() system call





write() system call





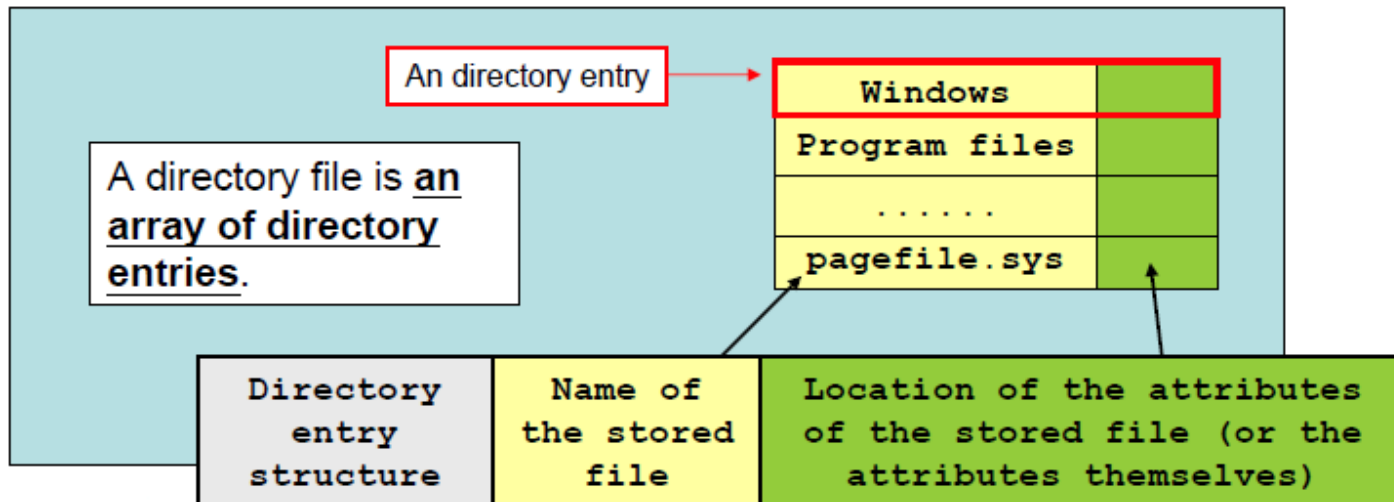
Directory

- A directory is **a file**.
 - Then, does it imply that it has **file attributes** and **file content**?

Answer: Sure

Answer: FS dependent

- How does a directory file look like?





Directory Traversal Process

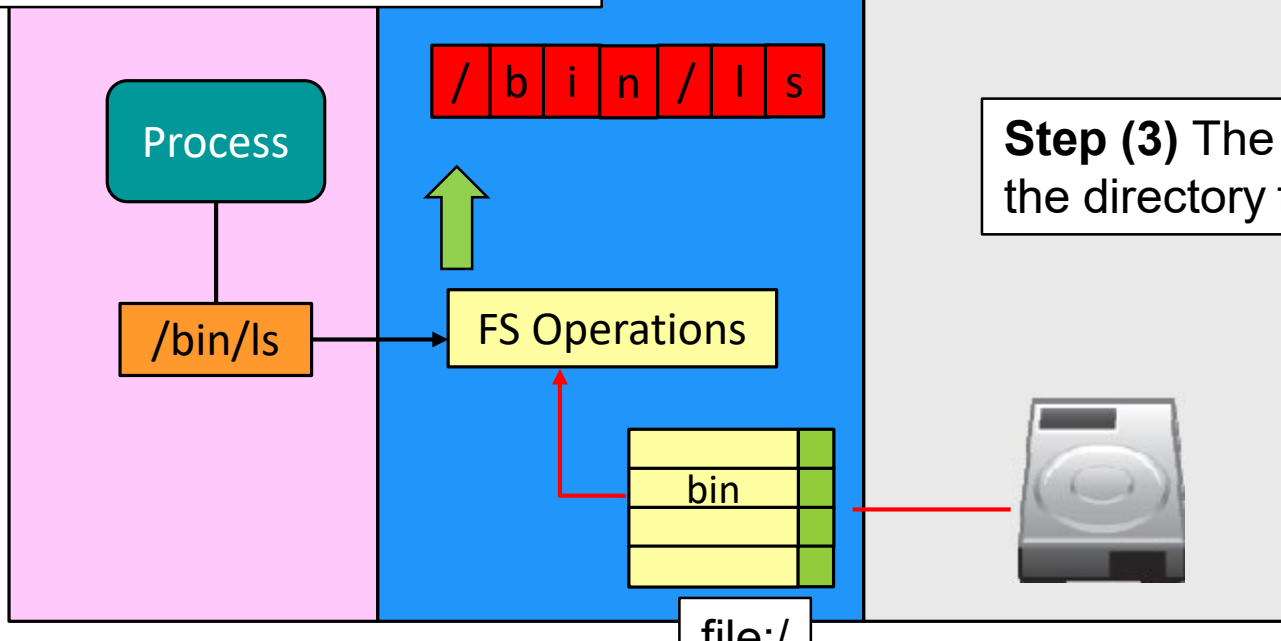
- How to locate a file using pathname?

Step (1) Suppose that the process wants to open the file `"/bin/ls"`.

The process then supplies the OS the unique pathname `"/bin/ls"`.

Step (2) The OS retrieves the directory file of the root directory `'/'`.

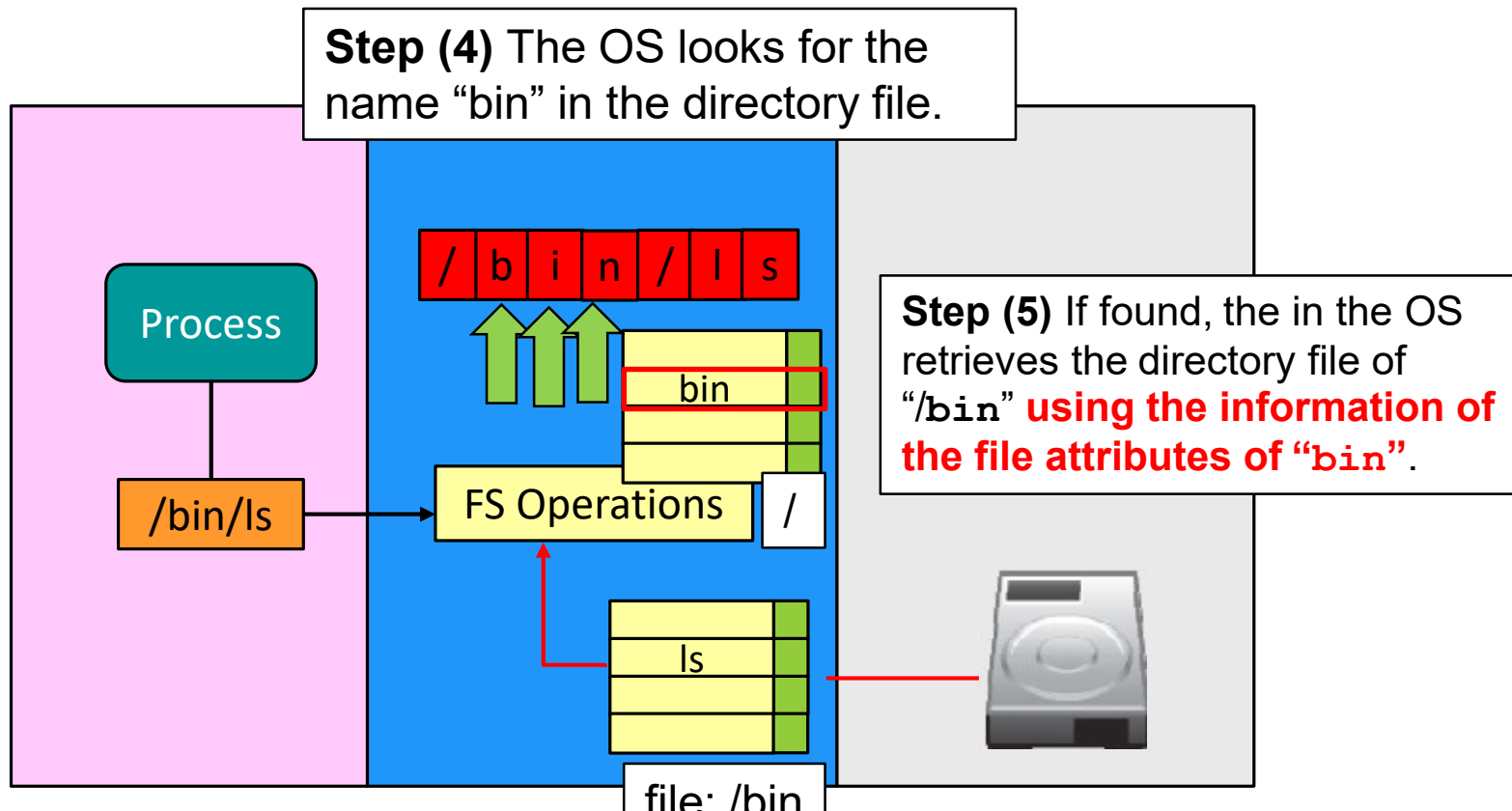
Step (3) The disk returns the directory file.





Directory Traversal Process

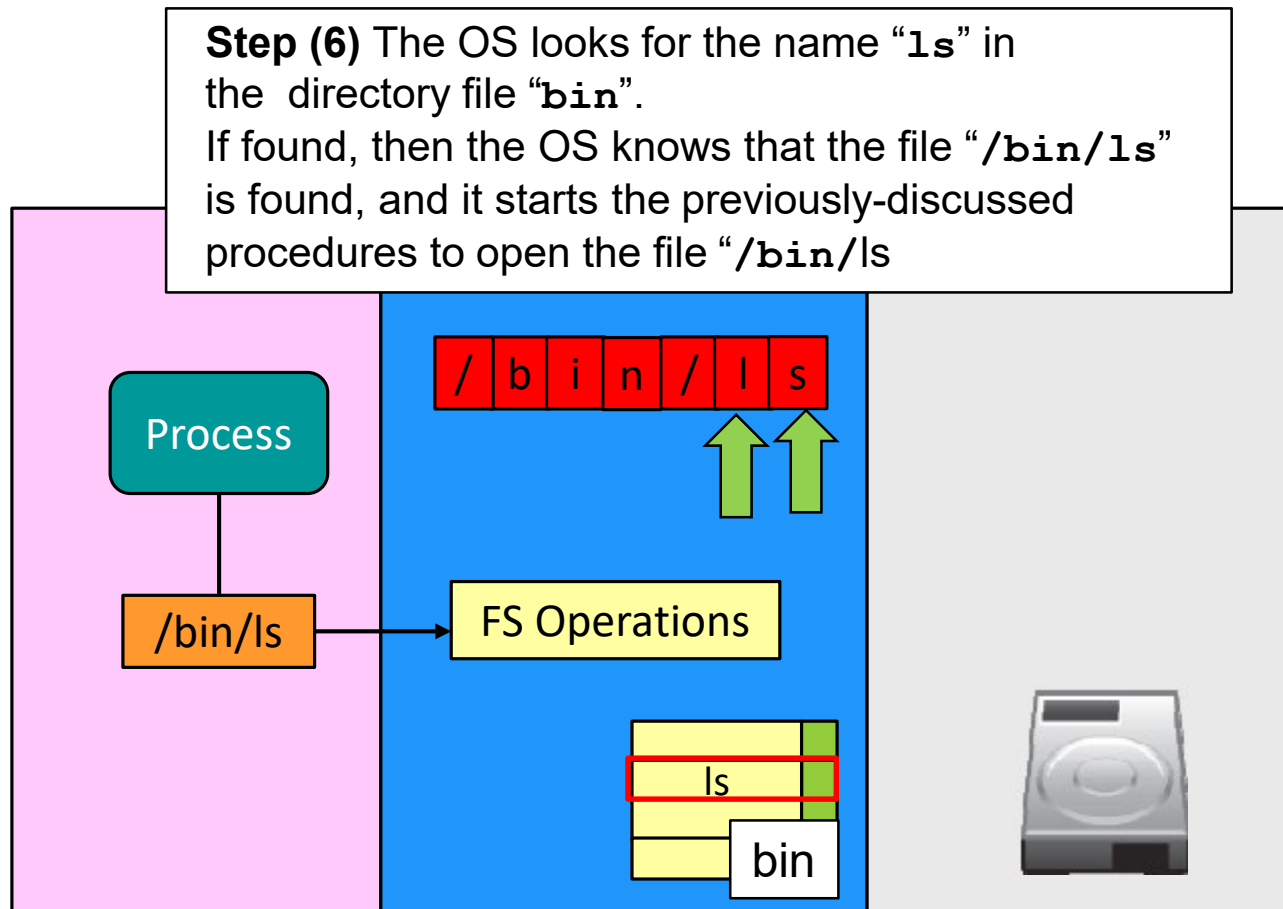
- How to locate a file using pathname?





Directory Traversal Process

- How to locate a file using pathname?





File Creation and Directory

- According to your experience, **what is the file creation?**
 - E.g., creating a file named “`test.txt`”?
 - “`touch test.txt`”?
 - “`vim test.txt`”, then type “`:wq`”?
 - “`cp [some filename] test.txt`”?
- The truth is:
File creation == Update of the directory file



Updating directory file

- When/how to update a directory file?

Creating a directory file	<code>syscall - mkdir()</code> ; Example program - <code>mkdir</code> .
Add an entry to the directory file	<code>syscall - open(), creat()</code> ; Example program - <code>cp, mv, etc.</code>
Remove an entry to the directory file	<code>syscall - unlink()</code> ; Example program - <code>rm</code> .
Remove a directory file	<code>syscall - rmdir()</code> ; Example program - <code>rmdir</code> .



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



典型文件系统



- 1 第 1 节: FAT 文件系统
 - FAT Volume
 - File Allocation System
 - Filenames on FAT Volumes



File Allocation Table (FAT) Volume

- A simple file system originally **designed for small disks and simple folder structures**.
- The FAT file system is named for its method of organization, the **file allocation table**, which resides at the beginning of the volume.
- To protect the volume, two copies of the table are kept, in case one becomes damaged.
- **The file allocation tables and the root folder must be stored in a fixed location** so that the files needed to start the system can be correctly located.



Structure of a FAT Volume

Boot sector	File allocation table 1	File allocation table 2 (duplicate)	Root directory	Other directories and all files
-------------	-------------------------	-------------------------------------	----------------	---------------------------------

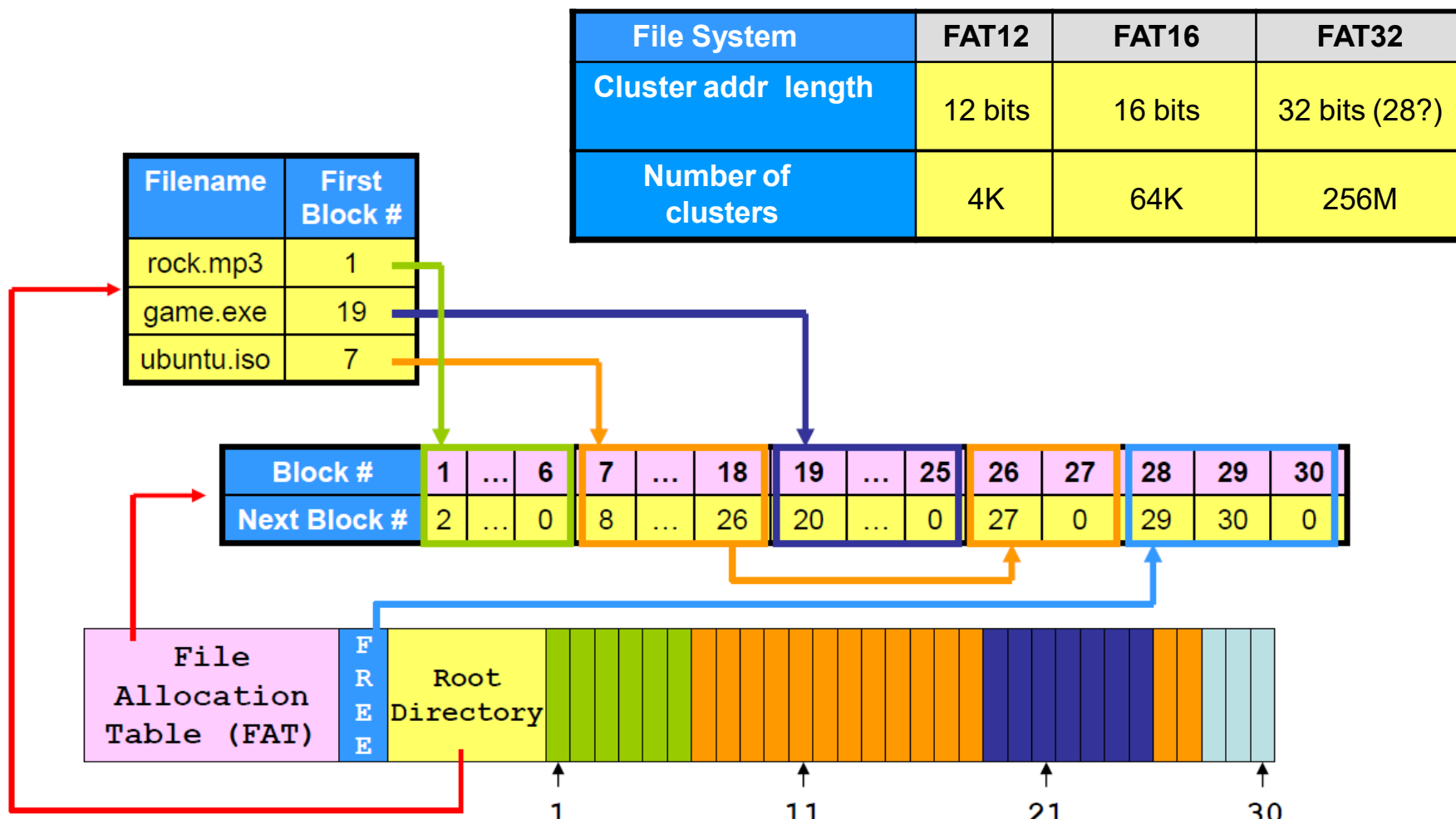
- Boot sector
- FAT1
- FAT2
- Root directory
- Other directories and all files



Recall on FAT allocation

- The layout

A block is named a **cluster**.





Typical layout of a FAT32 partition

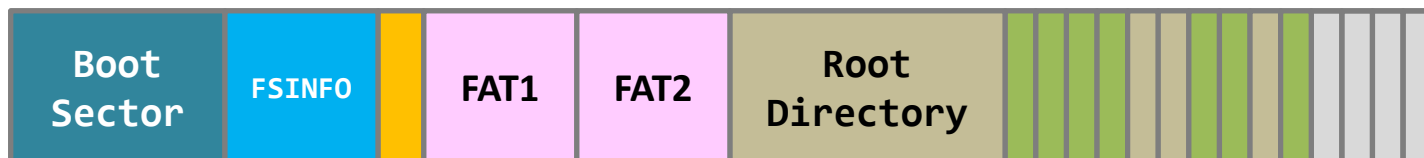
```
$ sudo mkfs.vfat -F32 /dev/ram0  
mkfs.fat 3.0.28 (2015-05-16)  
.....  
$ sudo dosfsck -v /dev/ram0
```

Format the disk, “-F32” means FAT32.

Read the information stored in the boot sector.

Running “**dosfsck**”, *DOS file system check*, on a FAT32 FS.

This program reads details from the **Boot Sector**.





Typical layout of a FAT32 partition

```
$ sudo mkfs.vfat -F32 /dev/ram0
mkfs.fat 3.0.28 (2015-05-16)
.....
$ sudo dosfsck -v /dev/ram0
fsck.fat 3.0.28 (2015-05-16)
```

Checking we can access the last sector of the filesystem

Boot sector contents:

System ID "mkfs.fat"

Media byte 0xf8 (hard disk)

512 bytes per logical sector

512 bytes per cluster

32 reserved sectors

First FAT starts at byte 16384 (sector 32)

2 FATs, 32 bit entries

516608 bytes per FAT (= 1009 sectors)

Root directory start at cluster 2 (arbitrary size)

Data area starts at byte 1049600 (sector 2050)

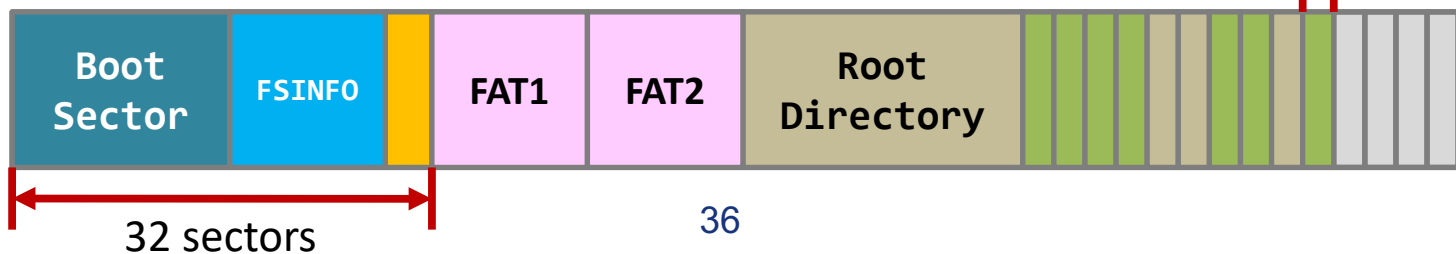
129022 data clusters (66059264 bytes)

.....

Details of the **Boot Sector**

The boot sector says:
A cluster is made of 1 sector.

One cluster size: 512
bytes in this case





Typical layout of a FAT32 partition

```
$ sudo mkfs.vfat -F32 /dev/ram0
mkfs.fat 3.0.28 (2015-05-16)
.....
$ sudo dosfsck -v /dev/ram0
fsck.fat 3.0.28 (2015-05-16)
```

Checking we can access the last sector of the filesystem

Boot sector contents:

System ID "mkdosfs"

Media byte 0xf8 (hard disk)

512 bytes per logical sector

512 bytes per cluster

32 reserved sectors

First FAT starts at byte 16384 (sector 32)

2 FATs, 32 bit entries

516608 bytes per FAT (= 1009 sectors)

Root directory start at cluster 2 (arbitrary size)

Data area starts at byte 1049600 (sector 2050)

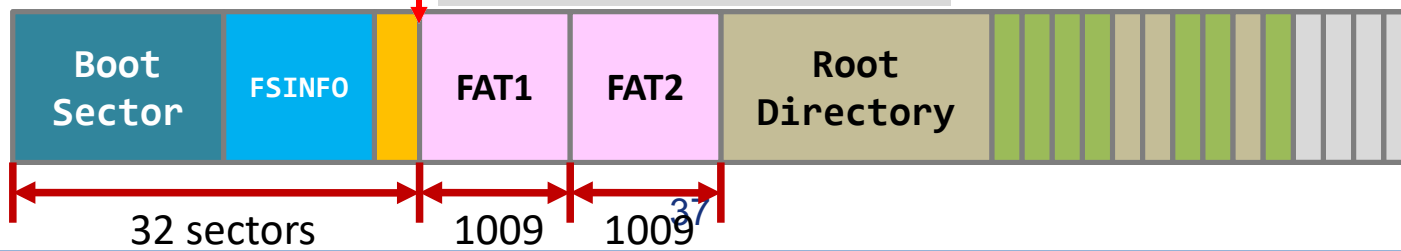
129022 data clusters (66059264 bytes)

.....

The boot sector says:
2 FATs and each of them is of
size **516,608 bytes**.

Number of FATs and the
length of each entry in a FAT.

Good! No slack space between
reserved sectors of the first FAT.





Typical layout of a FAT32 partition

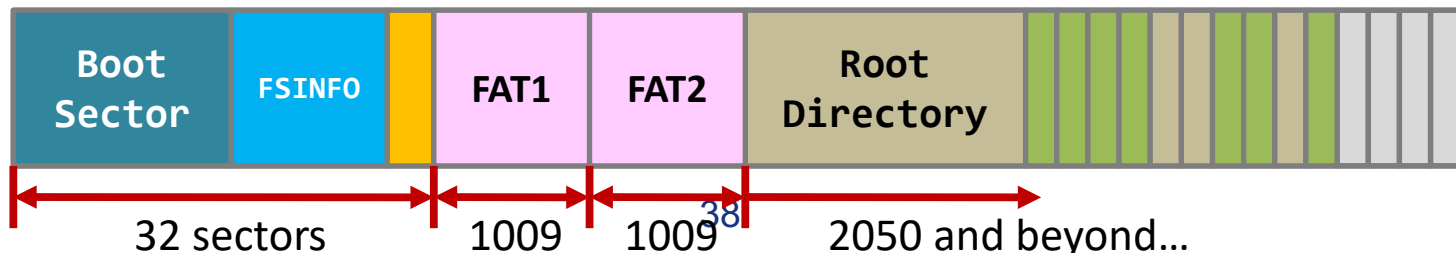
```
$ sudo mkfs.vfat -F32 /dev/ram0
mkfs.fat 3.0.28 (2015-05-16)
.....
$ sudo dosfsck -v /dev/ram0
fsck.fat 3.0.28 (2015-05-16)

Checking we can access the last sector of the filesystem
Boot sector contents:
System ID "mkdosfs"
Media byte 0xf8 (hard disk)
    512 bytes per logical sector
    512 bytes per cluster
    32 reserved sectors
First FAT starts at byte 16384 (sector 32)
    2 FATs, 32 bit entries
    516608 bytes per FAT (= 1009 sectors)
Root directory start at cluster 2 (arbitrary size)
Data area starts at byte 1049600 (sector 2050)
    129022 data clusters (66059264 bytes)
.....
```

The first data cluster is **Cluster #2** and it is usually, not always, the root directory.

Cluster #0 & #1 are reserved.

$$32 + 1009 \times 2 = 2050$$





Differences Between FAT Systems

System	Bytes Per Cluster Within File Allocation Table	Cluster limit
FAT12	1.5	Fewer than 4087 clusters.
FAT16	2	Between 4087 and 65526 clusters, inclusive.
FAT32	4	Between 65526 and 268,435,456 clusters, inclusive.

- FAT32 is a derivative of the File Allocation Table (FAT) file system that supports drives with over 2GB of storage.
- FAT32 drives can contain more than 65,526 clusters and results in more efficient space allocation on the FAT32 drive.



Directory Traversal

Step (1) Read the directory file of the root directory starting from **Cluster #2**.

“C:\windows” starts from Cluster #123.

```
c:\> dir c:\windows
```

```
.....
```

```
06/13/2012 2,033,216 explorer.exe
```

```
08/04/2015 169,120 notepad.exe
```

```
.....
```

```
c:\> _
```

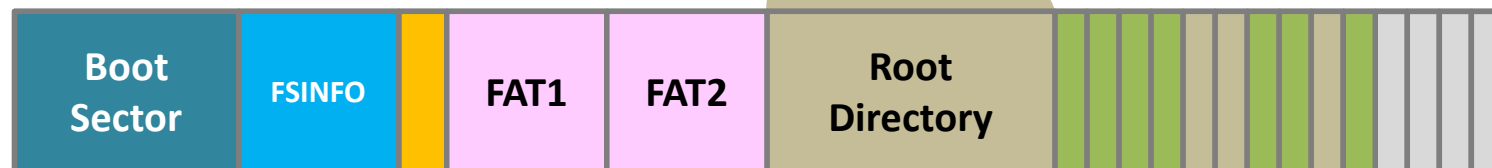
How does this work?

Cluster #2		
Filename	Attributes	Cluster #
.		?
..		?
.....		
windows		123

A directory entry

Check this out by yourself.

Whether those two directory entries exist or not.





Directory Traversal

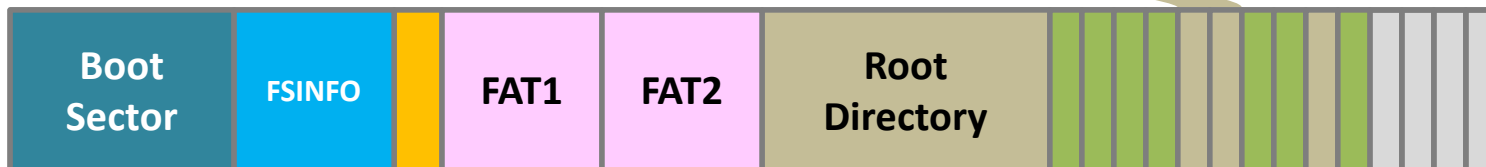
Step (2) Read the directory file of the “C:\windows” starting from **Cluster #123**.

```
c:\> dir c:\windows
.....
06/13/2012  2,033,216  explorer.exe
08/04/2015   169,120  notepad.exe
.....
c:\> _
```

How does this work?

Cluster #123		
Filename	Attributes	Cluster #
.		?
..		?
.....		
notepad.exe		456

But, where are the information, e.g., file size, modification time, etc?





Directory entry

- Directory entry is just a structure.

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden)
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

what?

Filename	Attributes	Cluster #
explorer.exe		32

How?

0	e	x	p	l	o	r	e	r	7
8	e	x	e	...	...	...	...	...	15
16	...	...	...	...	00	00	...	...	23
24	...	...	20	00	00	C4	0F	00	31

Note. This is the 8+3 naming convention.

8 characters for name +
3 characters for file extension



Directory entry

- Directory entry is just a structure.

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden)
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

what?

Filename	Attributes	Cluster #
explorer.exe		32

How?

0	e	x	p	l	o	r	e	r	7
8	e	x	e	...	...	...	...	...	15
16	...	...	...	...	00	00	...	...	23
24	...	...	20	00	00	C4	0F	00	31

How to calculate the first cluster address?



The file size...

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden)
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

what?

Filename	Attributes	Cluster #
explorer.exe		32

How?

0	e	x	p	l	o	r	e	r	7
8	e	x	e	...	...	...	...	...	15
16	...	...	...	...	00	00	...	...	23
24	...	...	20	00	00	C4	0F	00	31

So, what is the largest size of a file?

4G – 1 bytes



Directory entry

- Any problem with this design?

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden)
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

Note. This is the 8+3 naming convention.

8 characters for name +
3 characters for file extension

Example:

How to store the file:

"I_love_the_operating_system_course.txt"

How to store long
filename?



FAT series – LFN directory entry

- LFN: Long File Name.
 - In FAT32, the 8+3 naming convention is removed by...
 - Adding more entries to represent the filename

Directory file
LFN #3
LFN #2
LFN #1
Normal Entry

Each LFN entry represents 13 characters in Unicode, i.e., 2 bytes per character. Yet, the sequence is upside-down!

The normal directory entry is still there.



FAT series – LFN directory entry

Normal entry

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden)
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

LFN entry

Bytes	Description
0-0	Sequence Number
1-10	File name characters (5 characters in Unicode)
11-11	File attributes - always 0x0F
12-12	Reserved.
13-13	Checksum
14-25	File name characters (6 characters in Unicode)
26-27	Reserved
28-31	File name characters (2 characters in Unicode)



FAT series – LFN directory entry

- Filename:
“I_love_the_operating_system_course.txt”.

Byte 11 is always 0x0F to indicate that is a LFN.

LFN #3	436d	005f	0063	006f	0075	000f	0040	7200	Cm._.c.o.u...@r.
	7300	6500	2e00	7400	7800	0000	7400	0000	s.e...t.x...t...
LFN #2	0265	0072	0061	0074	0069	000f	0040	6e00	.e.r.a.t.i...@n.
	6700	5f00	7300	7900	7300	0000	7400	6500	g._.s.y.s...t.e.
LFN #1	0149	005f	006c	006f	0076	000f	0040	6500	.I._.l.o.v...@e.
	5f00	7400	6800	6500	5f00	0000	6f00	7000	_.t.h.e._...o.p.
Normal	495f	4c4f	5645	7e31	5458	5420	0064	b99e	I_LOVE~1TXT .d..
	773d	773d	0000	b99e	773d	0000	0000	0000	w=w=...w=.....



FAT series – LFN directory entry

This is the sequence number, and they are arranged in descending order.

The terminating directory entry has the sequence number **OR-ed with 0x40**.

Directory file

LFN #3: "m_cou" "rse.tx" "t"

LFN #2: "erati" "ng_sys" "te"

LFN #1: "I_lov" "e_the_" "op"

Normal Entry

LFN #3

43 6d 005f 0063 006f 0075 000f 0040 7200 Cm._.c.o.u...@r.
7300 6500 2e00 7400 7800 0000 7400 0000 s.e...t.x...t...

LFN #2

02 65 0072 0061 0074 0069 000f 0040 6e00 .e.r.a.t.i...@n.
6700 5f00 7300 7900 7300 0000 7400 6500 g._.s.y.s...t.e.

LFN #1

01 49 005f 006c 006f 0076 000f 0040 6500 .I._.l.o.v...@e.
5f00 7400 6800 6500 5f00 0000 6f00 7000 _t.h.e._...o.p.

Normal

495f 4c4f 5645 7e31 5458 5420 0064 b99e I_LOVE~1TXT .d..
773d 773d 0000 b99e 773d 0000 0000 0000 W=W=...W=.....



How to read a file?

Task: read "C:\windows\explorer.exe" sequentially.

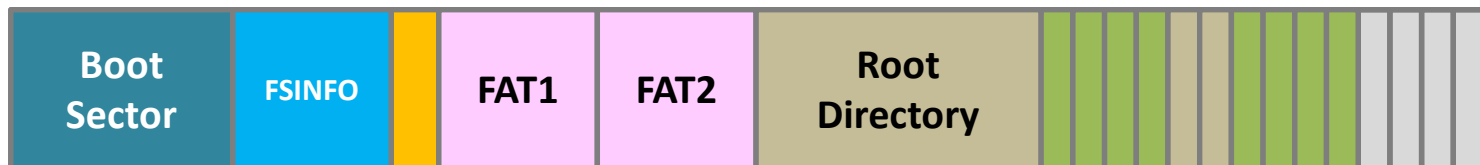
Suppose we already read out the directory entry...

You know the process of directory traversal, right?

Filename	Attributes	Cluster #
explorer.exe		32

Step 1. Read the content from Cluster #32.

Note. The **file size** may also help determining if the last cluster is reached (remember where it is stored?)





How to read a file?

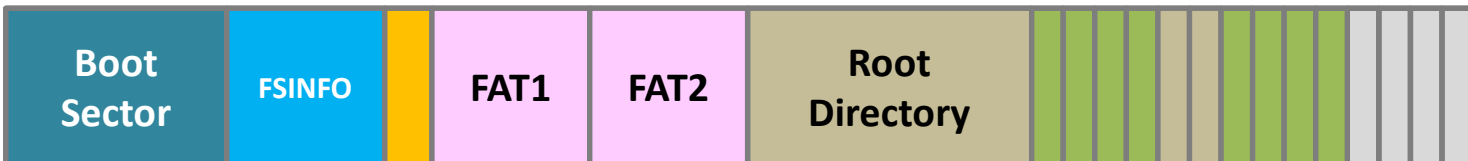
Task: read "C:\windows\explorer.exe" sequentially.

0	...
1	...
...	...
32	33
33	EOF
34	0
35	0

Filename	Attributes	Cluster #
explorer.exe		32

Step 1. Read the content from Cluster #32.
Note. The **file size** may also help determining if the last cluster is reached.

Step 2. Look for the next cluster and it is Cluster #33 (from the **FAT** table)





How to read a file?

Task: read "C:\windows\explorer.exe" sequentially.

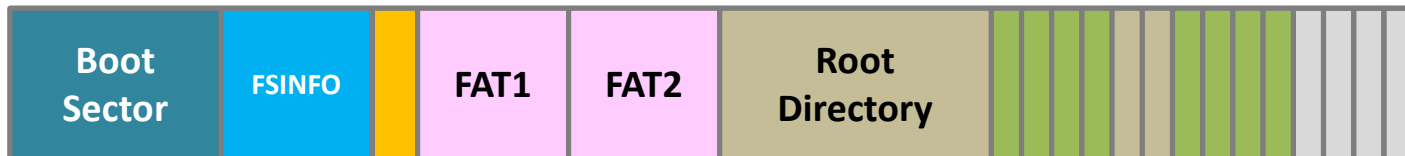
0	...
1	...
...	...
32	33
33	EOF
34	0
35	0

Filename	Attributes	Cluster #
explorer.exe		32

Step 3. Since the FAT has marked "EOF", we have reached the last cluster.

Note. The file size help determine **how many bytes to read** from the last cluster.

FAT entry structure??
Remember: 28bits are used to represent cluster number for FAT32





How to read a file?

Task: read "C:\windows\explorer.exe" sequentially.

0	...
1	...
...	...
32	33
33	EOF
34	0
35	0

Damaged = 0x0ffffff7

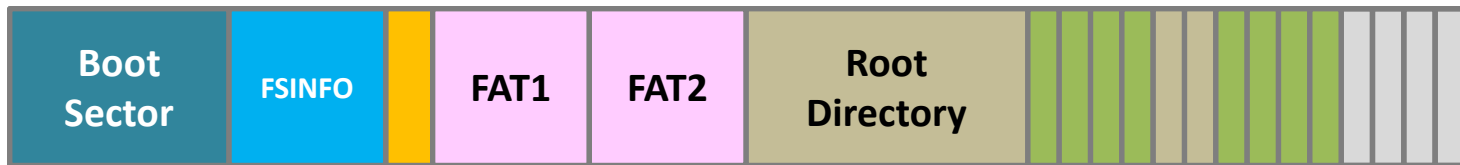
EOF >= 0x0ffffff8

Unallocated = 0x0

Filename	Attributes	Cluster #
explorer.exe		32

Step 3. Since the FAT has marked "EOF", we have reached the last cluster.

Note. The file size help determine **how many bytes to read** from the last cluster.





How to delete a file?

Task: delete "C:\windows\explorer.exe".

0	...
1	...
...	...
32	33
33	34
34	EOF
35	0



0	...
1	...
...	...
32	0
33	0
34	0
35	0

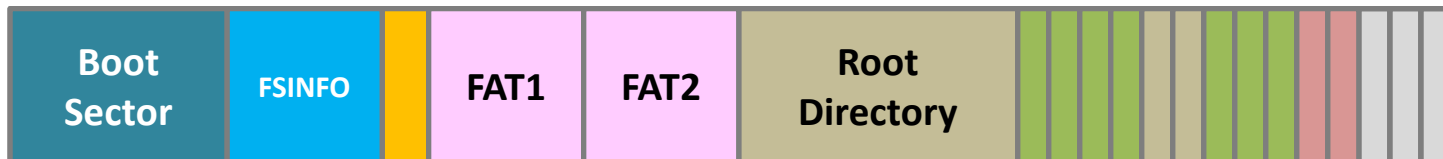
Filename	Attributes	Cluster #
explorer.exe		32

Step 1. De-allocate all the blocks involved. Update FSINFO and FATs.

FSINFO	
# of free clusters	3
Next free cluster #	35



FSINFO	
# of free clusters	6
Next free cluster #	32



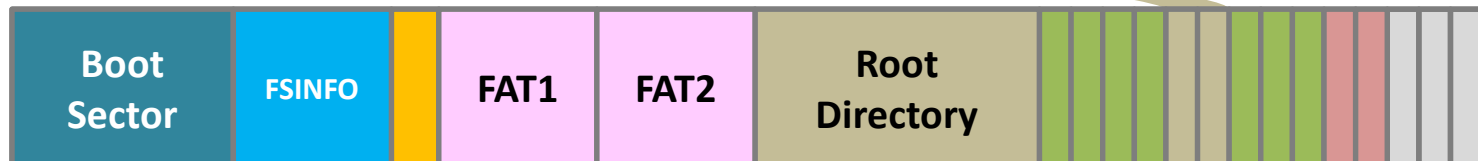


How to delete a file?

Task: delete "C:\windows\explorer.exe".

How about the directory entry

Cluster #123		
Filename	Attributes	Cluster #
.		?
..		?
explorer.exe		32
notepad.exe		456





How to delete a file?

Task: delete "C:\windows\explorer.exe".

Bytes	Description
0-0	1 st character of the filename (0x00 or 0xe5 means unallocated)

The first character becomes "0xE5".

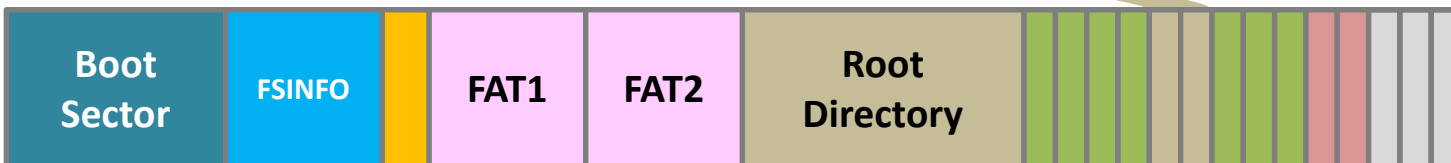
Cluster #123		
Filename	Attributes	Cluster #
.		?
..		?
_xplore.exe		32
notepad.exe		456

How about the directory entry

Step 2. Change the first byte of the directory entry to 0xE5.

LFN entries also receive the same treatment.

That's the end of deletion!





Really delete a file?

- Can you see that: **the file is not really removed from the FS layout?**
 - Perform a search in all the free space. Then, you will find all deleted file contents.
- “*Deleted data*” persists until the de-allocated clusters **are reused**.
 - This is an issue between performance (during deletion) and security.
- Any way(s) to delete a file **securely**?



How to delete a file “securely”?



Brute Force?

<http://www.ohgizmo.com/2009/06/01/manual-hard-drive-destroyer-looks-like-fun/>

What will the research community tell you?

<http://cdn.computerscience1.net/2006/Fall/lectures/8/articles8.pdf>



- **Extended File System (Ext2/3/4)**
 - Follow index-node allocation
 - Primary FS for Linux distribution
 - Ext4 was merged in the Linux 2.6.28 and released in 2008
 - Backward-compatible
 - For simplicity, we focus on Ext2/3
 - Features of Ext2/3/4
 - https://ext4.wiki.kernel.org/index.php/Main_Page
 - <http://e2fsprogs.sourceforge.net/ext2.html>



中山大學

SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

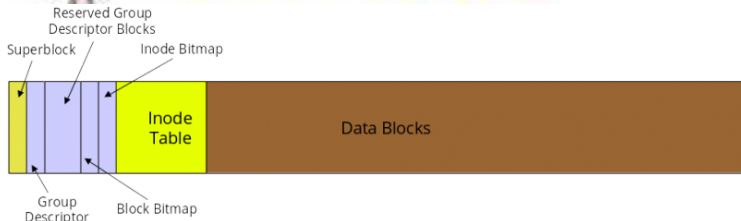
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



历史



MINIX 3



Ext 2 3 4

Ext4
File System

把

Linux 文件系统历史

尽管 EXT[1-4] 文件系统家族是为 Linux 编写的，但它的根源是 Minix 操作系统和 Minix 文件系统，它们早于 Linux 大约五年，于 1987 年首次发布。如果我们回顾一下 EXT 文件系统家族历史和技术从其 Minix 根源的演变，了解 EXT4 文件系统要容易得多。

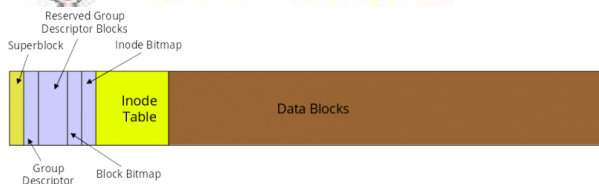
Reference:

OSTEP txtbook: Crash Consistency: FSCCK and Journaling; Ext4 slides from Mingming Cao; An introduction to Linux's EXT4 filesystem by David Both





历史



- Linus Torvalds 在编写原始的 Linux 内核时，他需要一个文件系统，但是不想编写一个文件系统。
- 拿来主义：直接采用 Andrew S. Tanenbaum 编写 Minix 文件系统



MINIX FS

- 索引节点区位图：表明哪些索引节点已使用和哪些索引节点空闲。
- 索引节点区：它在磁盘上有自己的空间。每个 inode 包含有关一个文件的信息，包括数据块的位置，即属于该文件的区域。
- 数据区位图：来跟踪使用和免费的数据区。
- 数据区：保存实际存储数据。



历史

Ext 2 3 4

- 最初的 EXT 文件系统（扩展）由 RémyCard 编写，并于 1992 年随 Linux 发行，以克服 Minix 文件系统的某些大小限制。

EXT FS

- 主要的结构更改是基于 Unix 文件系统（UFS）（也称为 Berkeley 快速文件系统（FFS））的文件系统的元数据。很少有关于 EXT 文件系统的公开信息可以验证，显然是因为它存在重大问题，并很快被 EXT2 文件系统所取代。

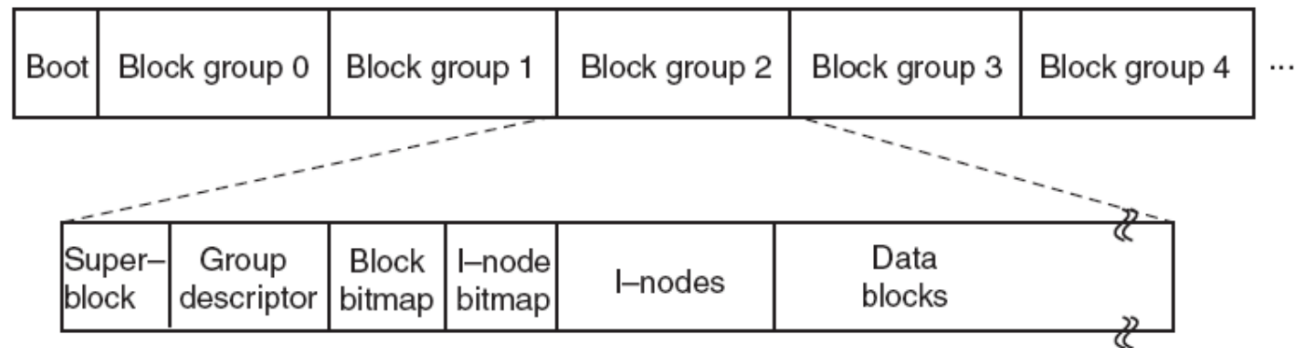


历史

Ext 2 3 4

- EXT2 文件系统具有与 EXT 文件系统基本相同的元数据结构，稳定可靠性有很大提升，被广泛用于 Linux 的早期版本。

EXT2 FS



- 在一个目录下的文件存储在同一个 block group 中
- 位于不同目录下的文件分布在不同的 block group 中
- 在突然掉电或机器死机后，文件系统恢复正常要花费很长时间

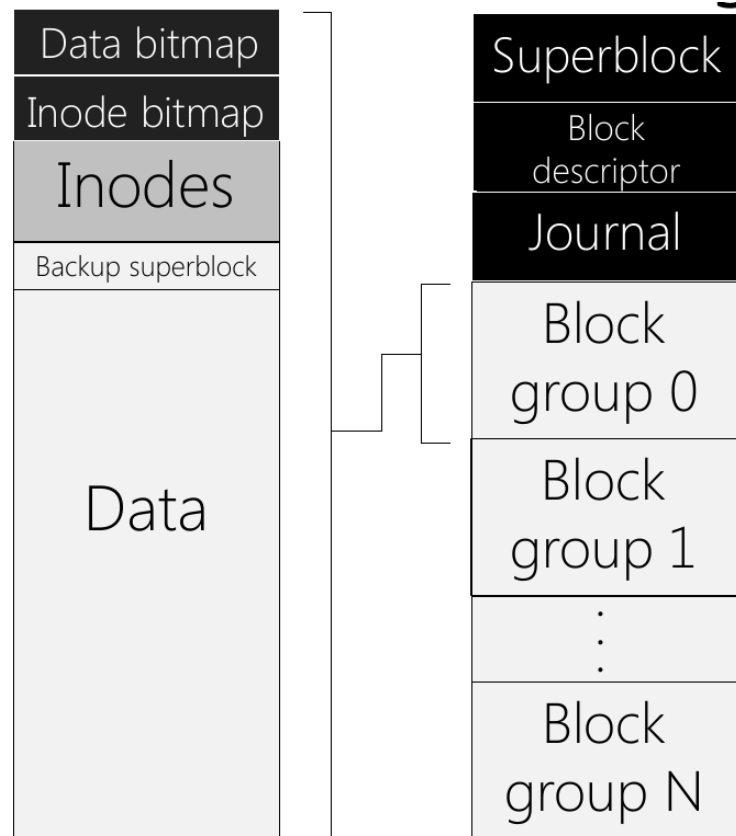


历史

Ext 2 3 4

- EXT3 的唯一目标是克服花费大量时间恢复异常文件系统的时间。EXT3 文件系统增加了 journal 机制，它预先记录了将对文件系统执行的更改。其余磁盘结构与 EXT2 中的相同。

EXT3 FS





历史

Ext4 File System

- EXT4 文件系统主要进一步改善了性能，可靠性和大容量支持

EXT4 FS

- 为了提高可靠性，添加了元数据和日志校验和。
- 为了满足各种关键任务要求，文件系统时间戳得到了改进，增加了纳秒精度时间间隔。
- 为提高容量和访问大容量文件的性能，把数据分配从固定块更改为扩展区



细节 – 支持大容量存储

Ext4
File System

- EXT4 的重要特征：
支持大容量存储和
快速恢复异常状态

EXT4 FS：支持大容量存储



- 一部蓝光 8K 3D 电影，512GB~1TB
- 1EB 文件系统大小，16TB 文件大小，子目录个数无限制



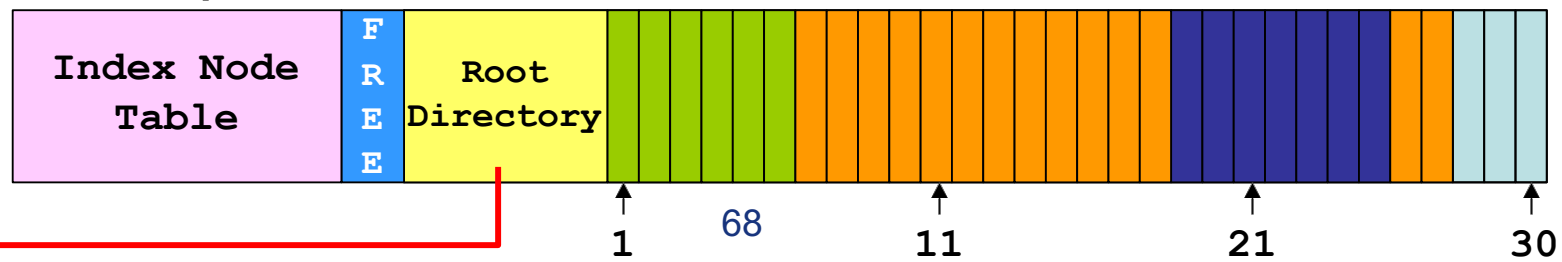
Index-node allocation

- Ext2/3 file systems follow the index-node allocation

Filename	Index Node #
rock.mp3	1
game.exe	2
ubuntu.iso	3

Index node #1		Index node #2		...		Index node #n-1	
	...		...				...
	...		...				...

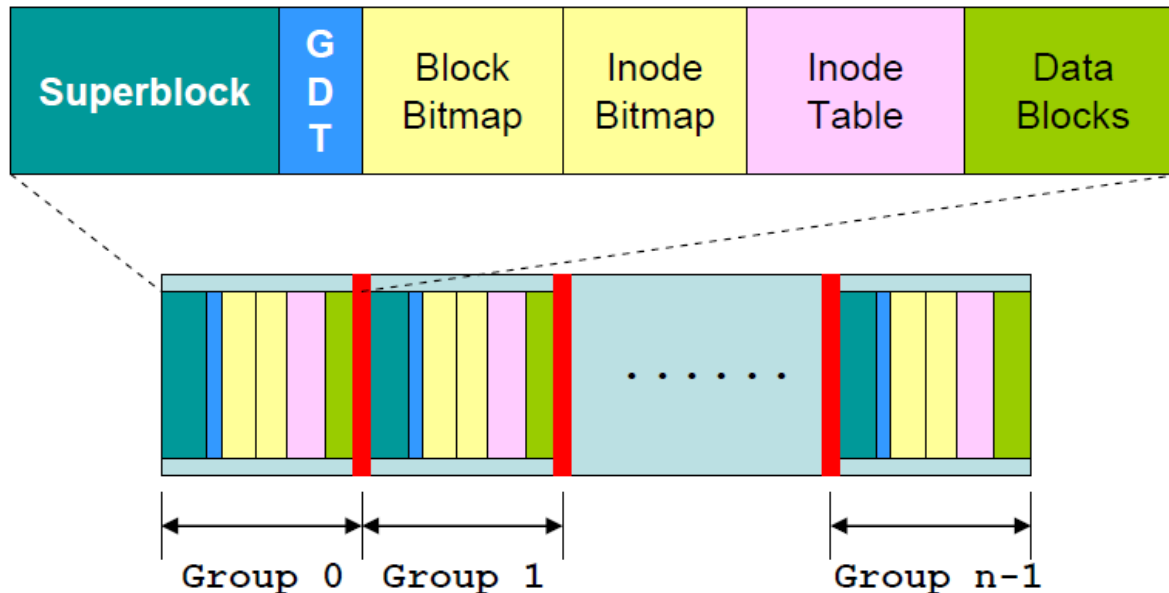
It is arranged as **an array**. So, looking up an index node will be fast.





Specific Layout

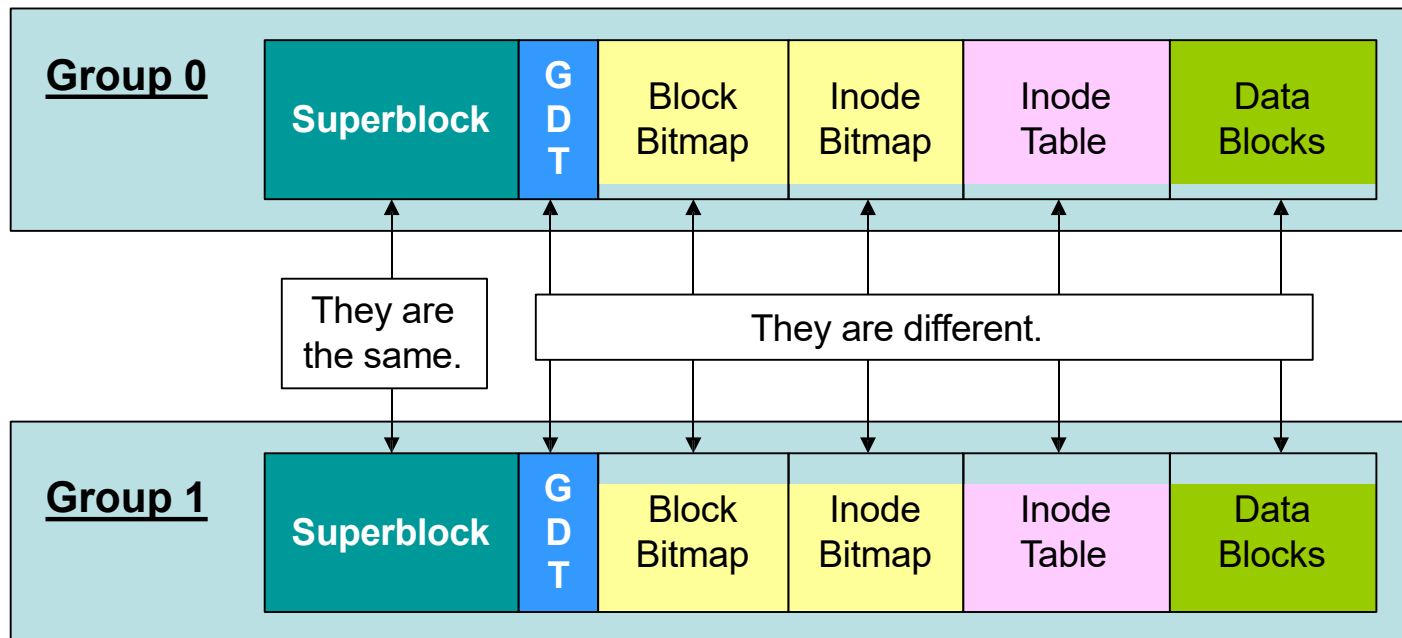
- The file system is not that simple...
 - it is divided into groups, and ...
 - every group has the same structure.





Specific Layout

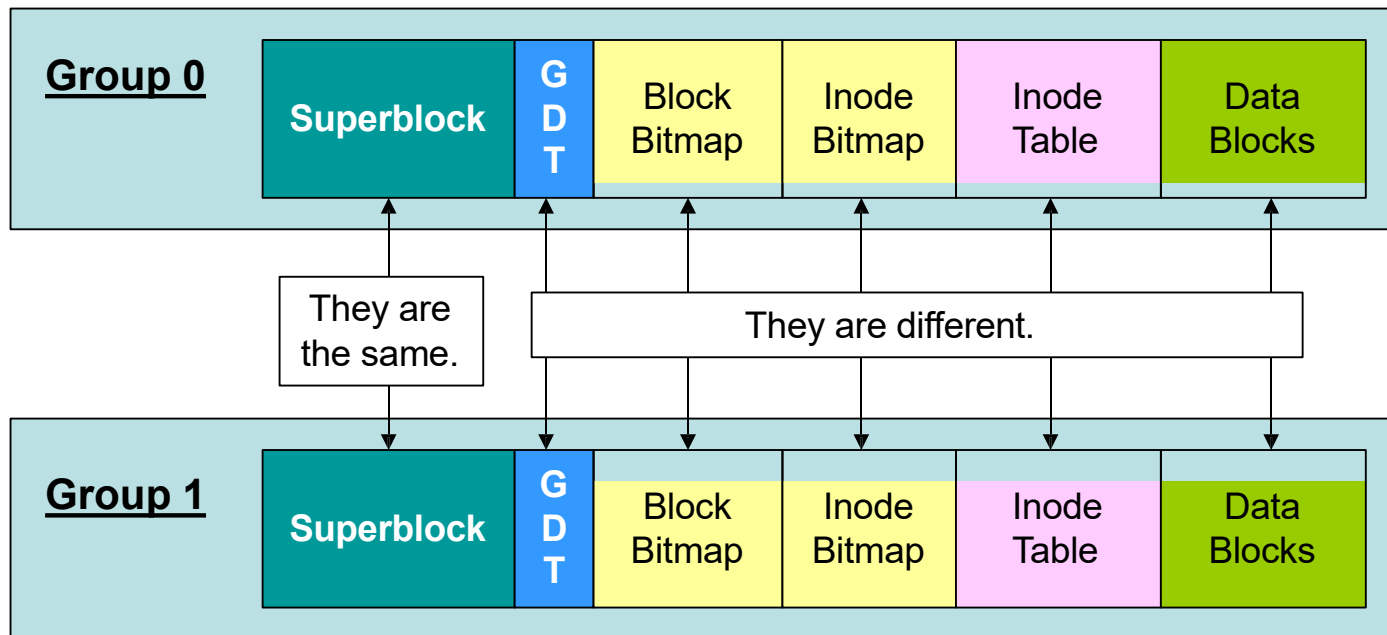
- The file system is not that simple...
 - it is divided into groups, and ...
 - every group has the same structure.





Specific Layout

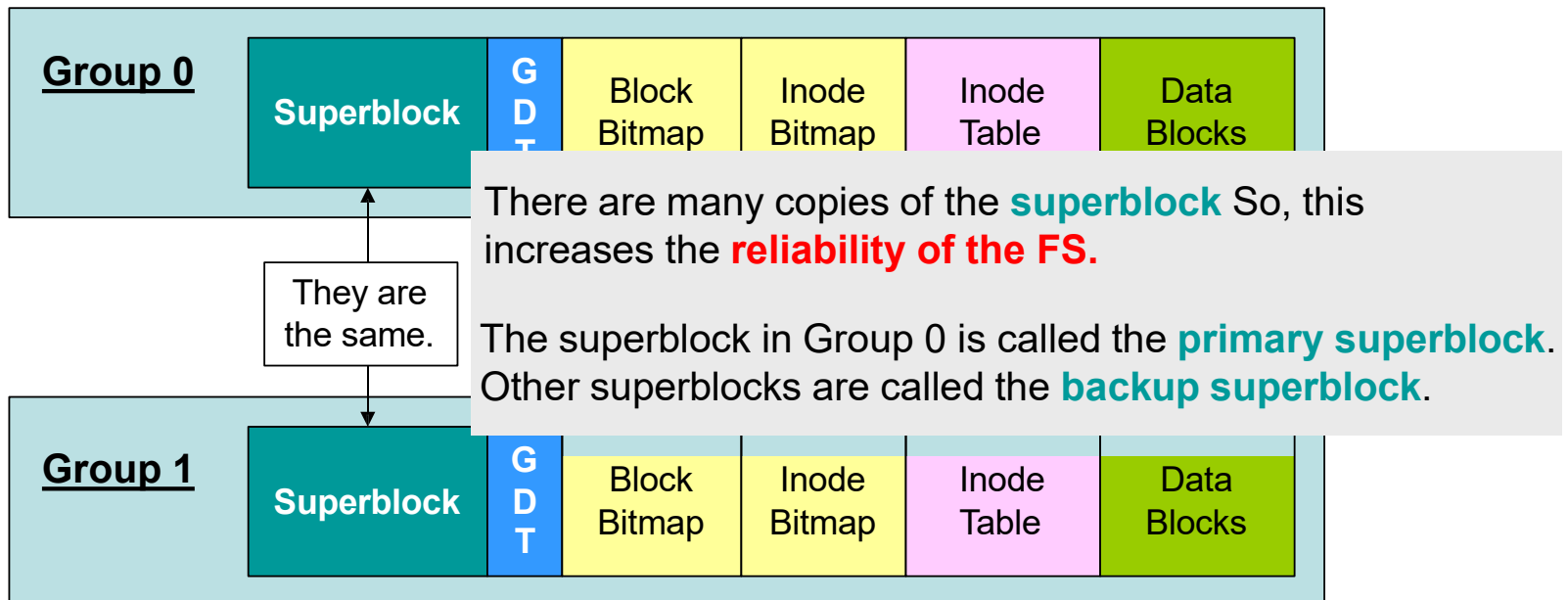
- Why doing so?
 - This is for **reliability and performance**.





Specific Layout

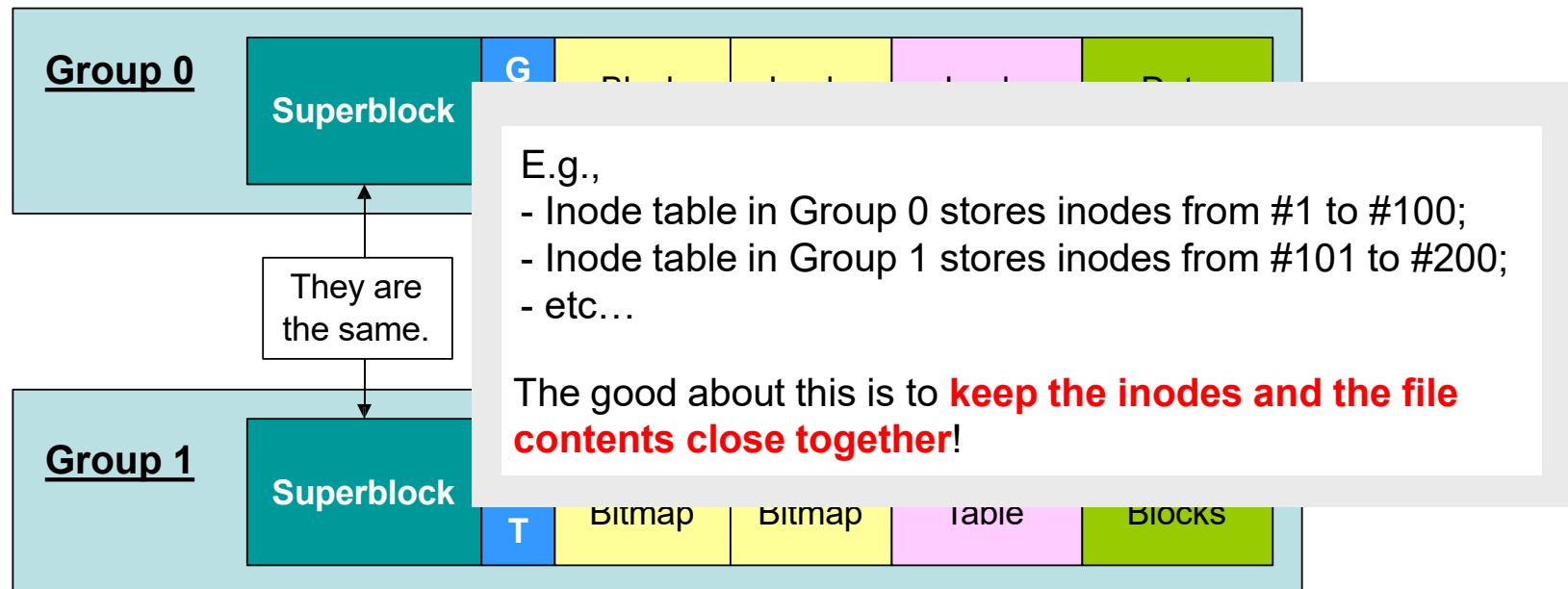
- Why doing so?
 - For **reliability**...





Specific Layout

- Why doing so?
 - For **performance...**



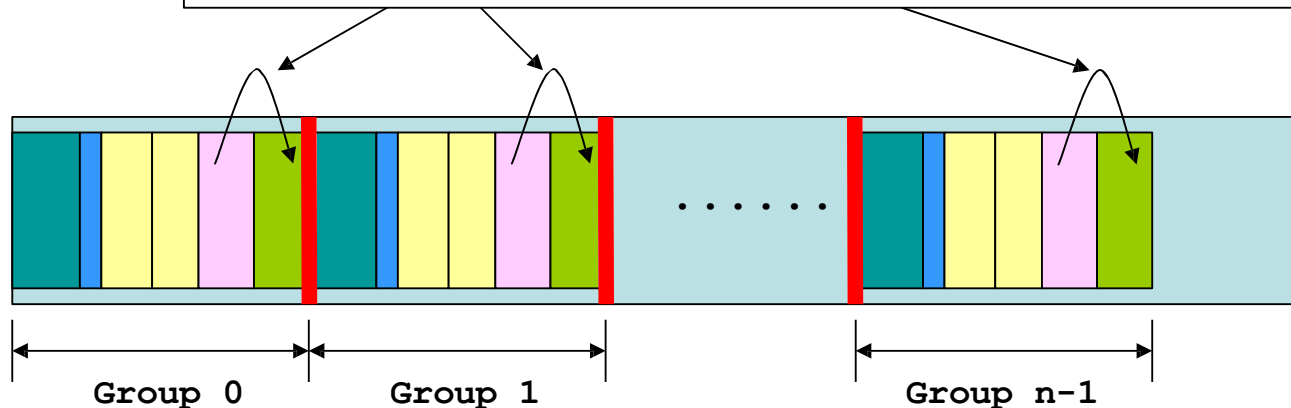


Specific Layout

- Why doing so?
 - For **performance...**

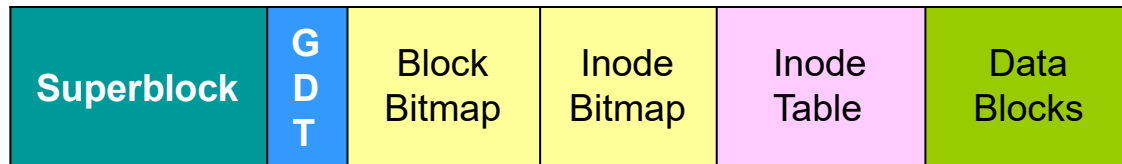
The inodes in a particular group will *usually* refer to the data blocks in the same group.

So, this keeps them close together in a physical sense. The storage device may be able to locate the data in a faster manner. (Remember the **principle of locality**?)

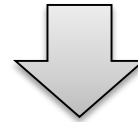




Layout in Each Group



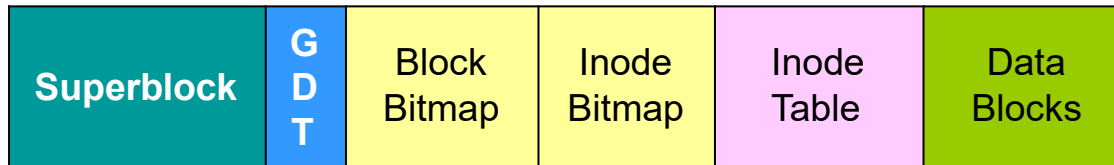
Superblock	Stores FS specific data.
------------	--------------------------



Total number of inodes in the system.
Total number of blocks in the system.
Number of reserved blocks
Total number of free blocks.
Total number of free inodes.
Location of the first block.
The size of a block.



Layout in Each Group

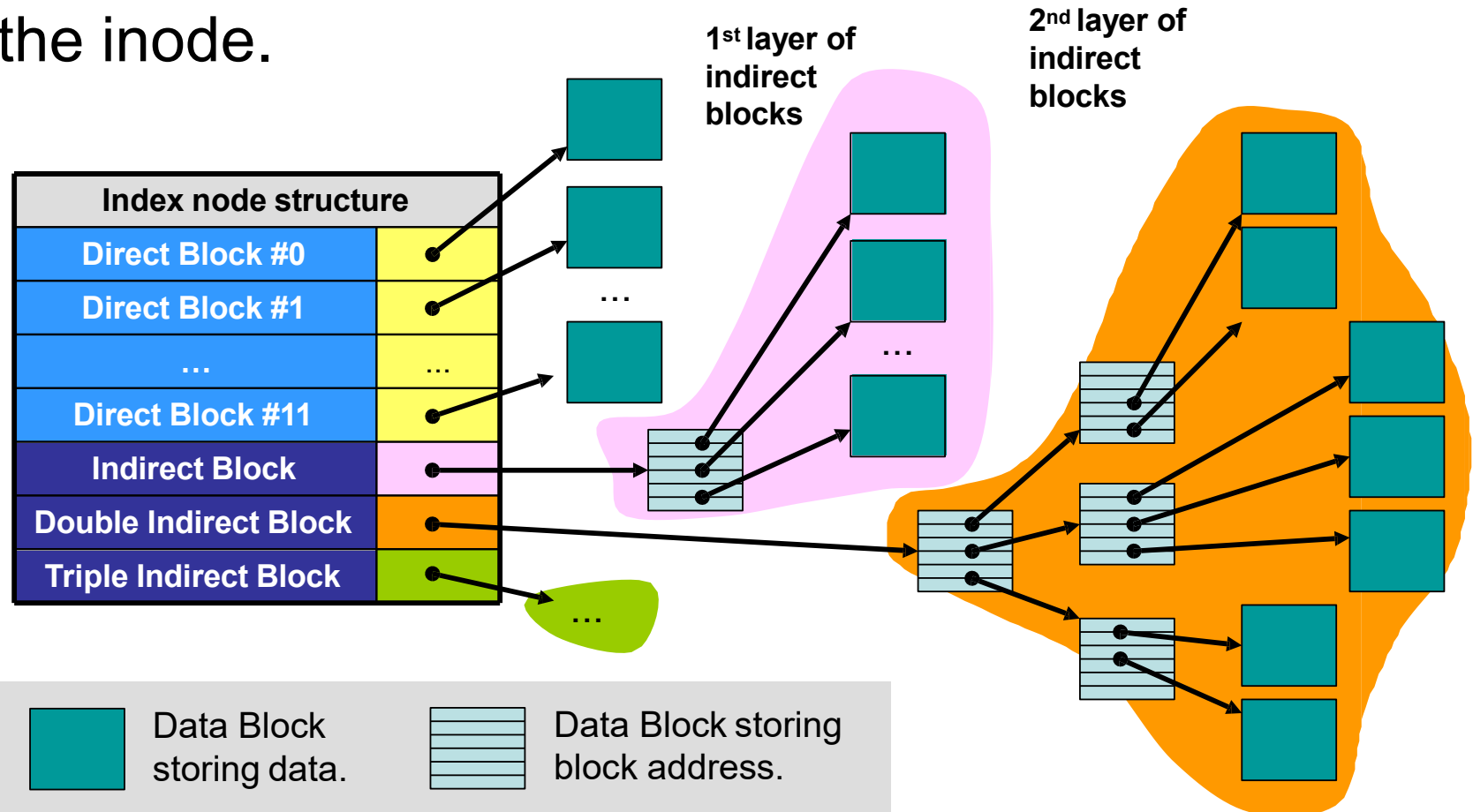


Superblock	Stores FS specific data. E.g., the total number of blocks, etc.
GDT – Group Descriptor Table	It stores: -The starting block numbers of the block bitmap, the inode bitmap, and the inode table. - Free block count, free inode count, etc...
Inode Table	An array of inodes ordered by the inode #.
Data Blocks	An array of blocks that stored files.
Block Bitmap	A bit string that represents if a block is allocated or not.
Inode Bitmap	A bit string that represents if an inode is allocated or not.



Inode Structure

- We know that...
 - The locations of the data blocks of a file are stored in the inode.





Inode Structure

Inode Structure (128 bytes long)	
Bytes	Value
0-1	File type and permission
2-3	User ID
4-7	Lower 32 bits of file sizes in bytes
8-23	Time information
24-25	Group ID
26-27	Link count
...	...
40-87	12 direct data block pointers
88-91	Single indirect block pointer
92-95	Double indirect block pointer
96-99	Triple Indirect block pointer
...	...
108-111	Upper 32 bits of file sizes in bytes

What are stored in inode besides block addresses?

An inode is the structure that stores every information about a file.

The locations of the data blocks



Inode Structure

Inode Structure (128 bytes long)	
Bytes	Value
0-1	File type and permission
2-3	User ID
4-7	Lower 32 bits of file sizes in bytes
8-23	Time information
24-25	Group ID
26-27	Link count
...	...
40-87	12 direct data block pointers
88-91	Single indirect block pointer
92-95	Double indirect block pointer
96-99	Triple Indirect block pointer
...	...
108-111	Upper 32 bits of file sizes in bytes

What is the maximum file size supported?

$$2^{64} - 1$$

$$= 16 \times 2^{30} \text{ Gbytes} - 1 \text{ byte}$$

Is this really the case?

Remember the dominating factor: $2^{4 \times 6}$

Block size	File size
1024B = 2^{10}	~16 Gbytes
4096B = 2^{12}	~4 Tbytes



Inode Structure

Inode Structure (128 bytes long)	
Bytes	Value
0-1	File type and permission
2-3	User ID
4-7	Lower 32 bits of file sizes in bytes
8-23	Time information
24-25	Group ID
26-27	Link count
...	...
40-87	12 direct data block pointers
88-91	Single indirect block pointer
92-95	Double indirect block pointer
96-99	Triple Indirect block pointer
...	...
108-111	Upper 32 bits of file sizes in bytes

What is link count?

We will talk about it later

Where is the file name?

Let us take a look at the directory structure



Link File

- Can we allow a file to have multiple names and be accessed by several paths?
- How to create shortcuts?

Example use in Linux

```
# ls /dir1/12.jpg
12.jpg
# ln /dir1/12.jpg /my_link
# _
```

```
# ls /dir1/12.jpg
12.jpg
# ln -s /dir1/12.jpg /my_link
# _
```

These are called **hard link** and **symbolic link**



Link File – what is a hard link?

- A **hard link is a directory entry** pointing to an existing file.
 - **No new file content is created!**

```
# ls /dir1/12.jpg
12.jpg
# ln /dir1/12.jpg /my_link
# _
```

A new directory entry is created.

Directory: /dir1

Inode #	...	Filename
123	...	.
2	...	..
5,086	...	12.jpg

Directory: /

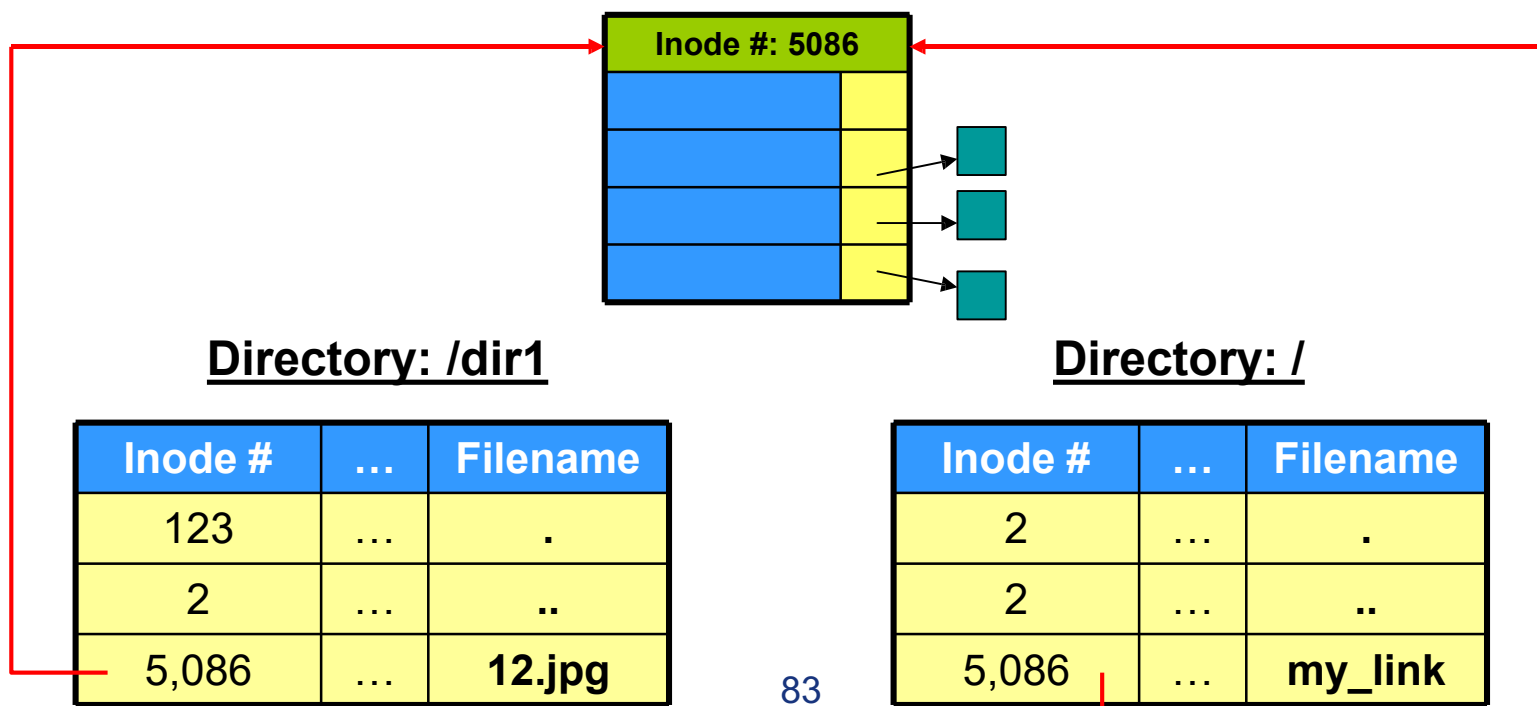
Inode #	...	Filename
2	...	.
2	...	..
5,086	...	my_link



Link File – what is a hard link?

- Conceptually speaking, this **creates a file with two pathnames.**

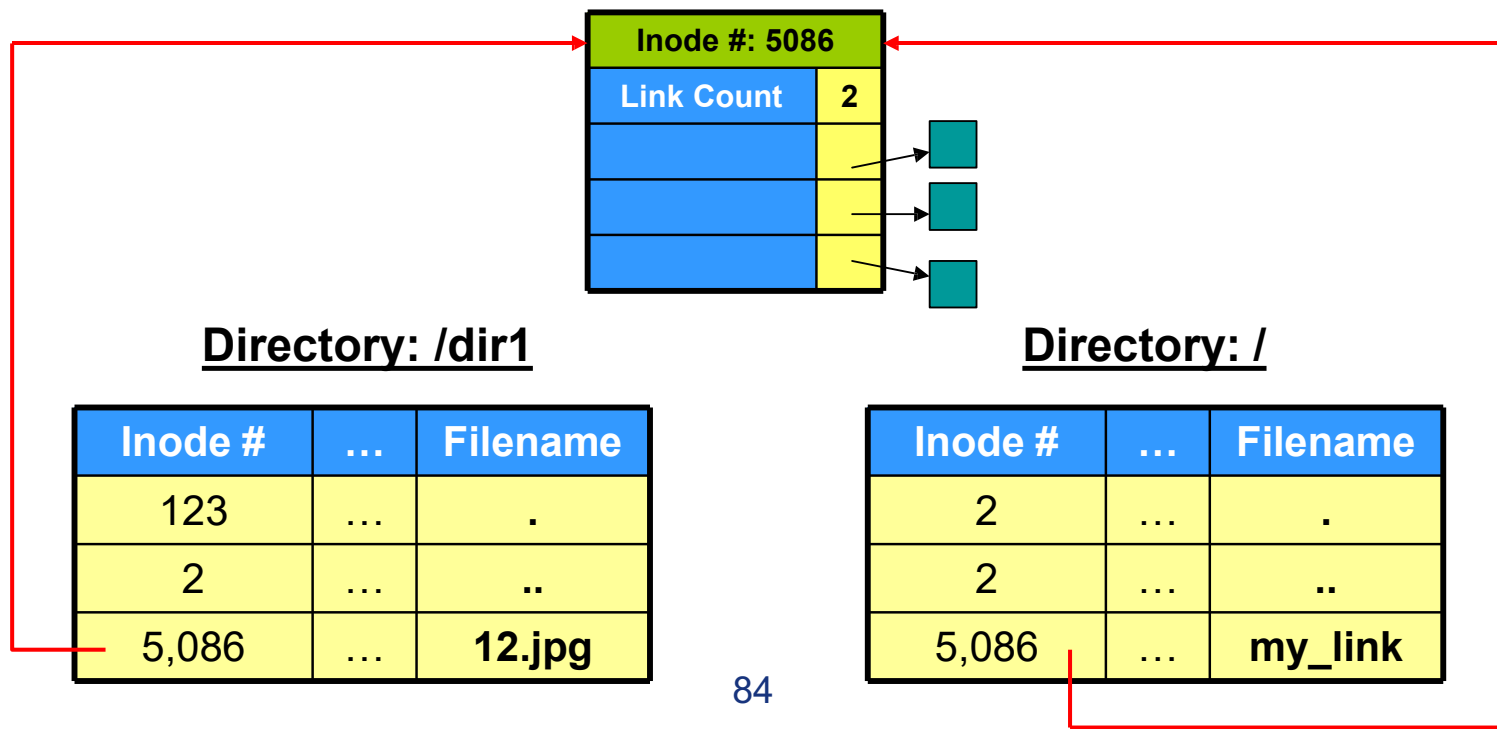
How to maintain this info.





Link File – what is a link count?

- There is a field called **link count** in an inode.
 - It stores the **number of directory entries** pointing to the inode.





Link File – showing the link counts

- Special hard links
 - The directory “.” is a hard link to itself.
 - The directory “..” is a hard link to the parent directory

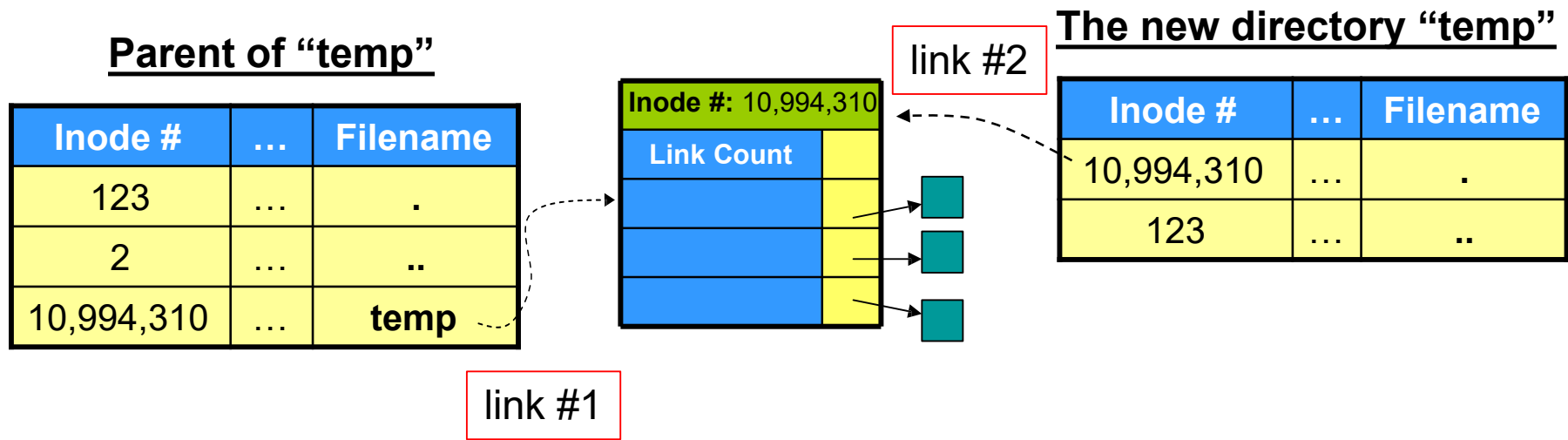
```
# ls -l /
total 124
drwxr-xr-x  2 root root  4096 2015-11-15 08:07 bin
drwxr-xr-x  4 root root  4096 2015-11-11 09:25 boot
drwxr-xr-x 17 root root 14520 2015-11-23 17:58 dev
drwxr-xr-x 165 root root 12288 2015-11-23 17:58 etc
drwxr-xr-x  6 root root  4096 2015-06-21 14:23 home
.....
```

What does this large number imply?

This implies “/etc” has a lot of sub-directories.



Link File – showing the link counts



- When a directory is created, **the initial link count is always 2**. Why?

```
# mkdir temp
# stat temp
  File: `temp'
  Size: 4096      Blocks: 8      IO Block: 4096   directory
Device: 804h/2052d  Inode: 10994310  Links: 2
.....
```



Link File – decrementing the link count?

- Back to the previous hard link example...

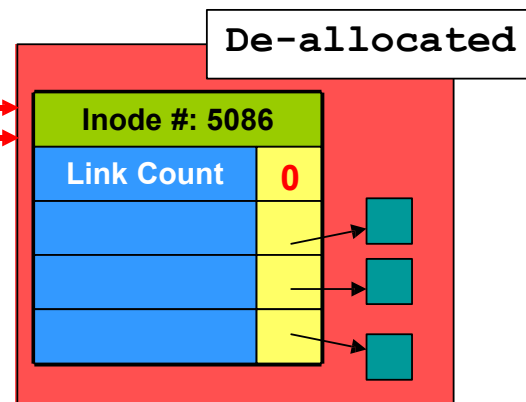
Directory: /dir1

Inode #	...	Filename
123	...	.
2	...	..
5,086	...	12.jpg

```
# ls /dir1/12.jpg
12.jpg
# ln /dir1/12.jpg /my_link
# rm /dir/12.jpg
# rm /my_link
```

Directory: /

Inode #	...	Filename
2	...	.
2	...	..
5,086	...	my_link





Link File – what is a symbolic link?

- A symbolic link **is a file**.
 - Unlike the hard link, **a new inode is created** for each symbolic link.
 - It stores the pathname (shortcut)

```
# ls /dir1/12.jpg
12.jpg
# ln -s /dir1/12.jpg /my_link
# ls -l /mylink
/mylink -> /dir1/12.jpg
#
```

A new directory entry is created.

Directory: /dir1

Inode #	...	Filename
123	...	.
2	...	..
5,086	...	12.jpg

Directory: /

Inode #	...	Filename
2	...	.
2	...	..
6,120	...	my_link

Another
inode

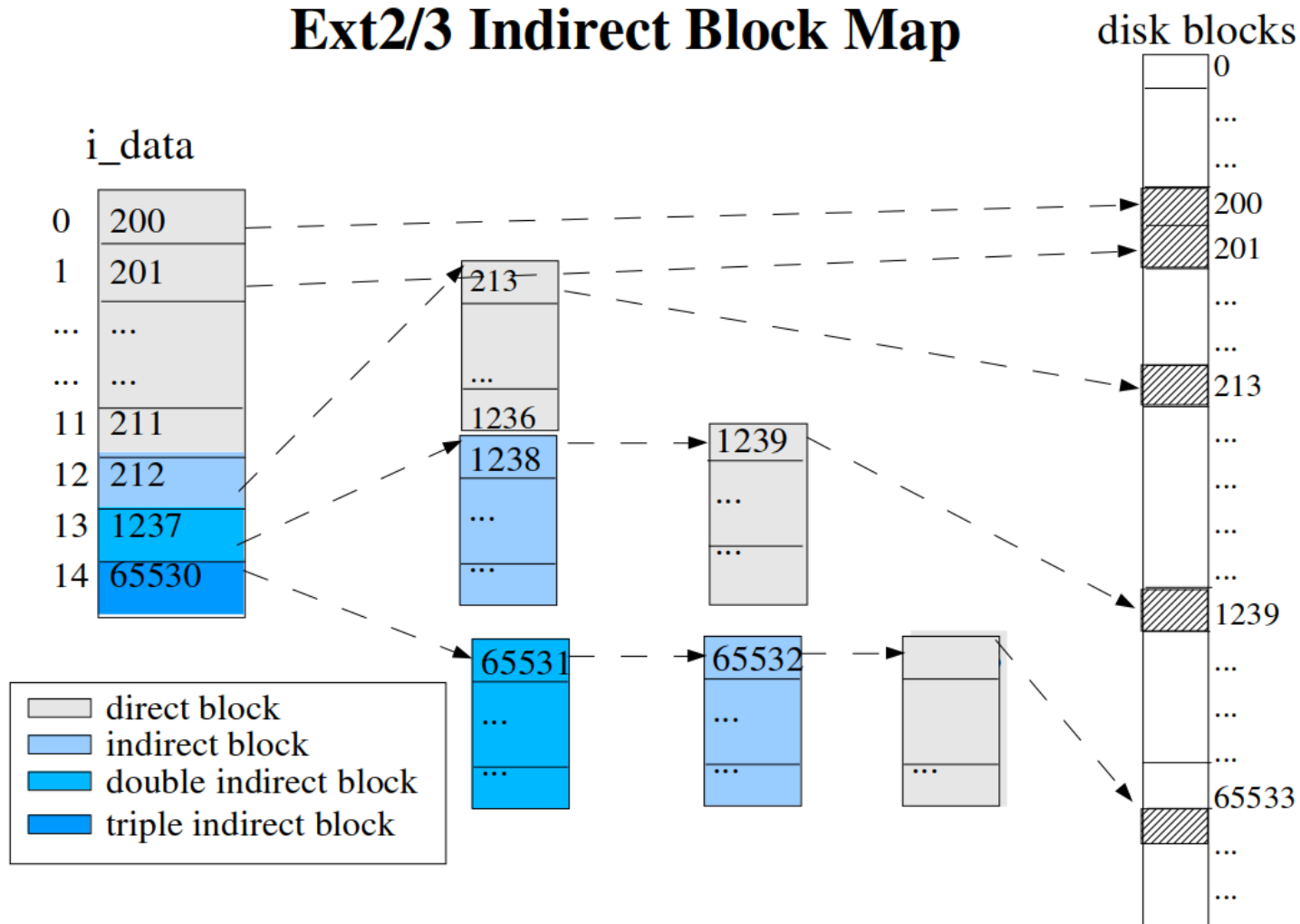
88





细节 – 支持大容量存储

Ext2/3 Indirect Block Map





细节 – 支持大容量存储

extent: 一段连续的储存块

logical	length	physical
0	1000	200

```
struct ext4_extent {  
    __le32 ee_block;    /* first logical block extent covers */  
    __le16 ee_len;      /* number of blocks covered by extent */  
    __le16 ee_start_hi; /* high 16 bits of physical block */  
    __le32 ee_start;    /* low 32 bits of physical block */  
};
```

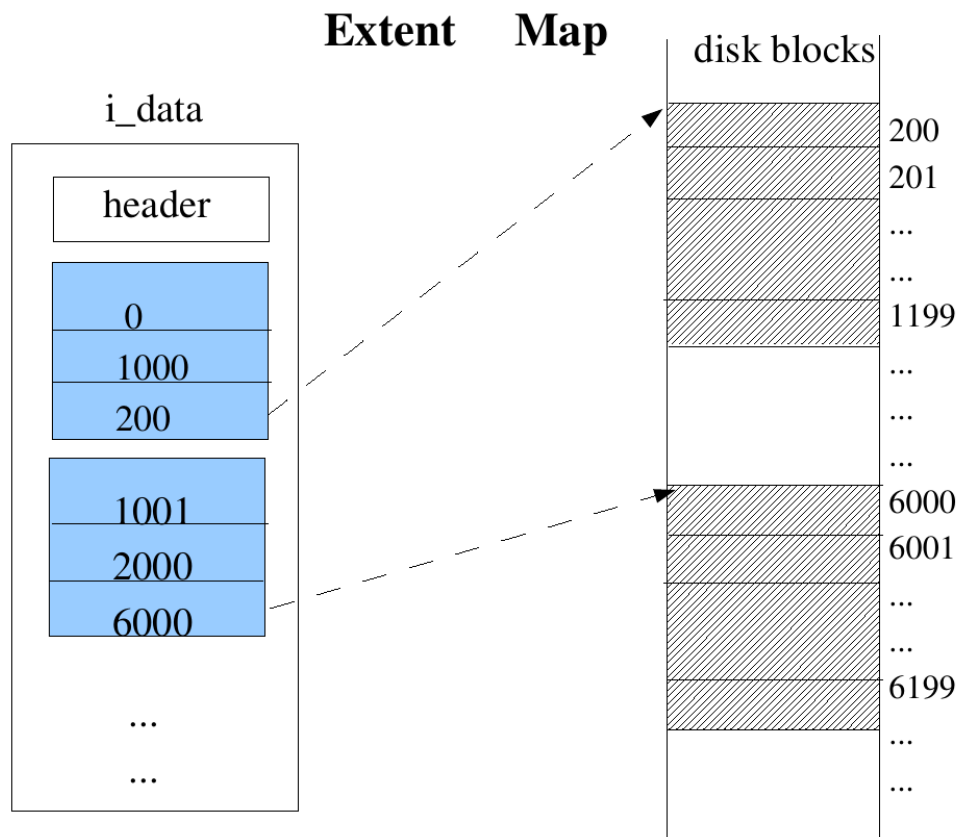


细节 – 支持大容量存储

extent: 一段连续的储存块

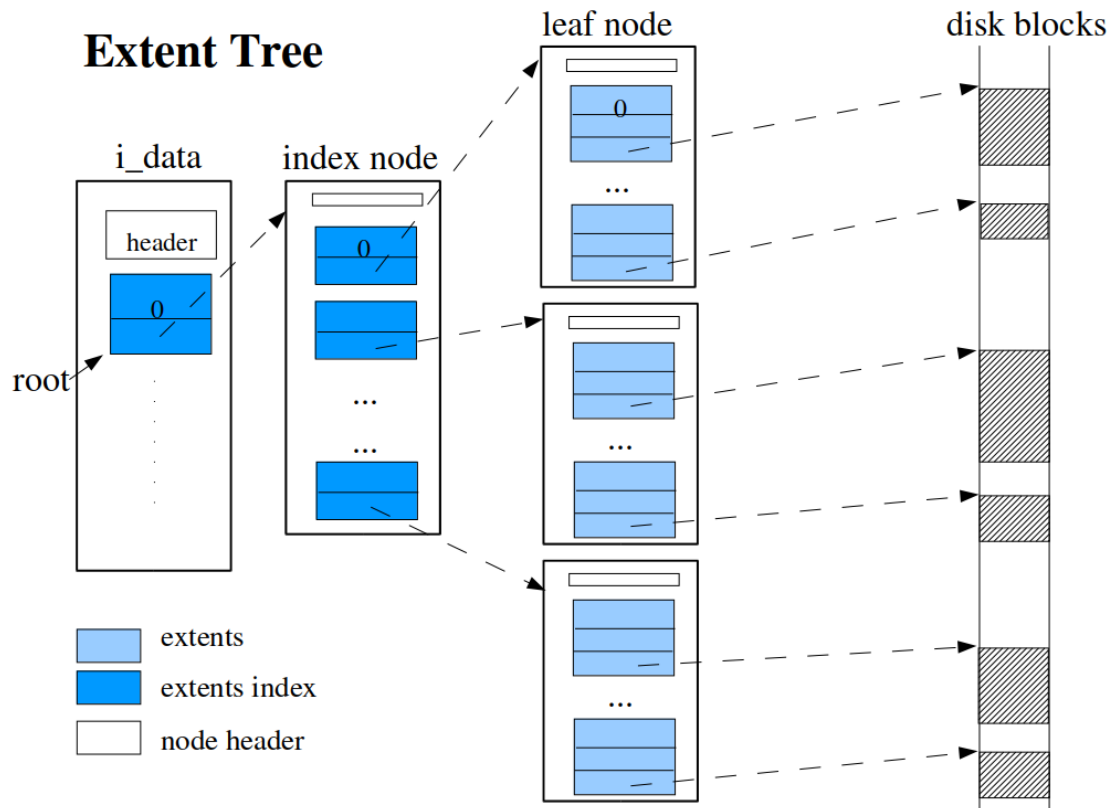
logical	length	physical
0	1000	200

```
struct ext4_extent {
    __le32 ee_block; /* first logical block extent covers */
    __le16 ee_len; /* number of blocks covered by extent */
    __le16 ee_start_hi; /* high 16 bits of physical block */
    __le32 ee_start; /* low 32 bits of physical block */
};
```





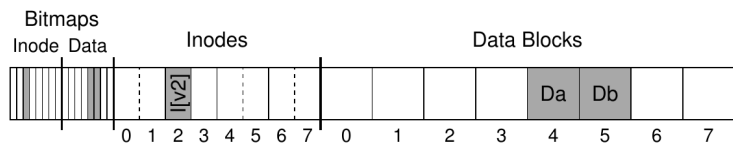
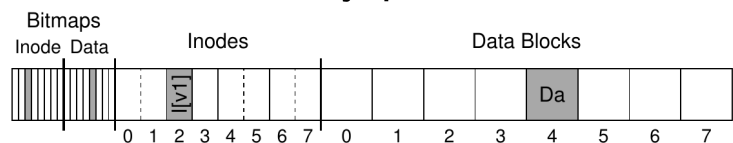
细节 – 支持大容量存储





细节 – 支持恢复异常 – FSCK

EXT4: 恢复异常文件系统 for crash-consistency problem



```
permissions : read-write
size        : 2
pointer     : 4
pointer     : 5
pointer     : null
pointer     : null
```

解决方法 1 : File System Checker (fsck)

- Superblock: 文件系统大小大于已分配的块数
- Free blocks: bitmap v.s. Free blocks
- Inode state/links: 有效的类型字段/链接计数
- Duplicates: 两个 inode 指向同一 data
- Bad blocks: 指针显然指向超出其有效范围
- Directory: “.” 和 “..” 是头两项

太慢



先是windows的：

FAT16 最大文件2GB 最大分区2GB

FAT32 最大文件4GB 最大分区128GB (如果用磁盘管理分区，最大32GB)

NTFS 最大文件2TB 最大分区2TB

Linux可支持多种操作系统，只介绍EXT2和EXT3

Ext2

最大文件大小: 1TB

最大文件极限: 仅受文件系统大小限制

最大分区/文件系统大小: 4TB

最大文件名长度: 255 字符

缺省最小/最大块大小: 1024/4096 字节

缺省inode分配: 每4096字节为1

在强制FS检查前的最大装载: 20(可配置)

Ext3

最大文件大小: 2TB

最大文件极限: 仅受文件系统大小限制

最大分区/文件系统大小: 4TB

最大文件名长度: 255 字符

缺省最小/最大块大小: 1024/4096 字节

缺省inode分配: 每4096字节为1

在强制FS检查前的最大装载: 20(可配置)



最大文件大小: 16TB

最大分区/文件系统大小: 1EB

(1EB=1024PB=1024*1024TB=1024*1024*1024GB)

单个目录下的子目录数目: 无限

ReiserFS

最大文件大小: 1TB

最大文件极限: 32k 目录, 42 亿文件

最大分区/文件系统大小: 4TB

最大文件名长度: 255 字符

JFS

最小文件系统大小 16 MB

最大文件大小: 受体系结构限制

最大文件极限: 受文件系统大小限制

缺省最小/最大块大小: 1024/4096 字节

缺省inode 分配: 动态

LVM

最大逻辑卷大小 从使用4 Mb 盘区的256 Gb

到使用较大PE 的1 Pb

最大逻辑卷数: 256

最大逻辑组数: 99

每PV 的最大PE 数: 65534

缺省物理盘区大小: 4 Mb



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢